

CodeFab Interpreter 프로젝트

CONTENTS

목차

CHAPTER 1

Code Fab 프로젝트 개요

CHAPTER 2

Assembler Unit 제작 - 1,2,3,4

CHAPTER 3

Checker Unit 제작

CHAPTER 4

Executor Unit 제작

CHAPTER 5

테스트용 스크립트

CHAPTER 6

팀 프로젝트 진행

Chapter 01

CodeFab Interpreter 프로젝트 개요

Custom 언어를 실행하는 interpreter 제작하기

Code Fab이란?

이곳에서 말하는 Code Fab이란?

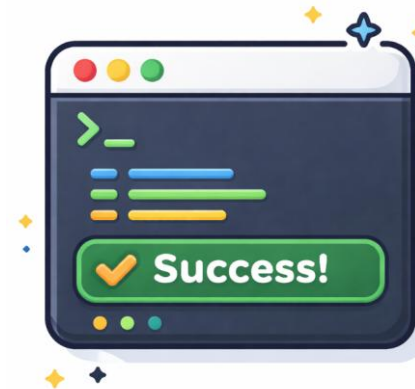
- 인터프리터(Interpreter) 를 의미
- 파이썬의 Interpreter로 비유될 수 있음
- Code를 넣으면 공장처럼 동작된다는 의미로, Code Fab 이라고 이름 지음



스크립트



LangFactory



실행결과

프로젝트 목표 - 1

프로젝트 목표

1. Custom Language 개발

팀 전용 Custom한 Language를 제작

2. Code Fab (Interpreter) 개발

Custom Language로 동작되는 인터프리터 제작 후 실행 되게끔 함

1. Custom Language 제작

팀에서
Custom한
프로그래밍
언어를 만든다.

2. Code Fab (Interpreter) 동작 순서

문법을
트리구조
로 만들기



오류검증



트리구조대로
실행하기

프로젝트 목표 - 2

3. Prompt Shell 제작

CLI Shell을 제작하여,

Custom Language 언어를 사용해서 즉시 결과를 확인할 수 있음

3. CLI 기반 Prompt Shell 예시

```
C:\Users\jeong>python
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
>>> a = 5
>>> b = 10
>>> print(a + b)
15
>>>
```

프로젝트 진행 방향

1일 ~ 2일차

- Custom Language 제작
- Code Fab 제작
- Prompt shell 제작

3일 ~ 4일차

- 기능 추가 예정

5일차

- 프로젝트 발표

좋지 않은 Custom Language

30 lines (28 sloc) | 678 Bytes

```
1  #include <iostream>
2  #define e using
3  #define ee namespace
4  #define eee std
5  #define eeee ;
6  #define eeeee int
7  #define eeeeeee main
8  #define eeeeeee (
9  #define eeeeeeee )
10 #define eeeeeeee while
11 #define eeeeeeeee true
12 #define eeeeeeeee {
13 #define eeeeeeeee }
14 #define eeeeeeeeeee cout
15 #define eeeeeeeeeee cerr
16 #define eeeeeeeeeee <<
17 #define eeeeeeeeeee 'e'
18 #define eeeeeeeeeee return
```

```
19
20 e ee eee eeee
21 eeeee eeeee eeeee eeeeeee
22 eeeeeeeee
23 eeeeeeeee eeeee eeeeeeeee eeeeeee
24 eeeeeeeee
25 eeeeeeeee eeeeeeeee eeeeeeeee
26 eeeeeeeee eeeeeeeee eeeeeee
27 eeeeeeeee
28 eeeeeeeee eeeeeeeee
29 eeeeeeeee
```



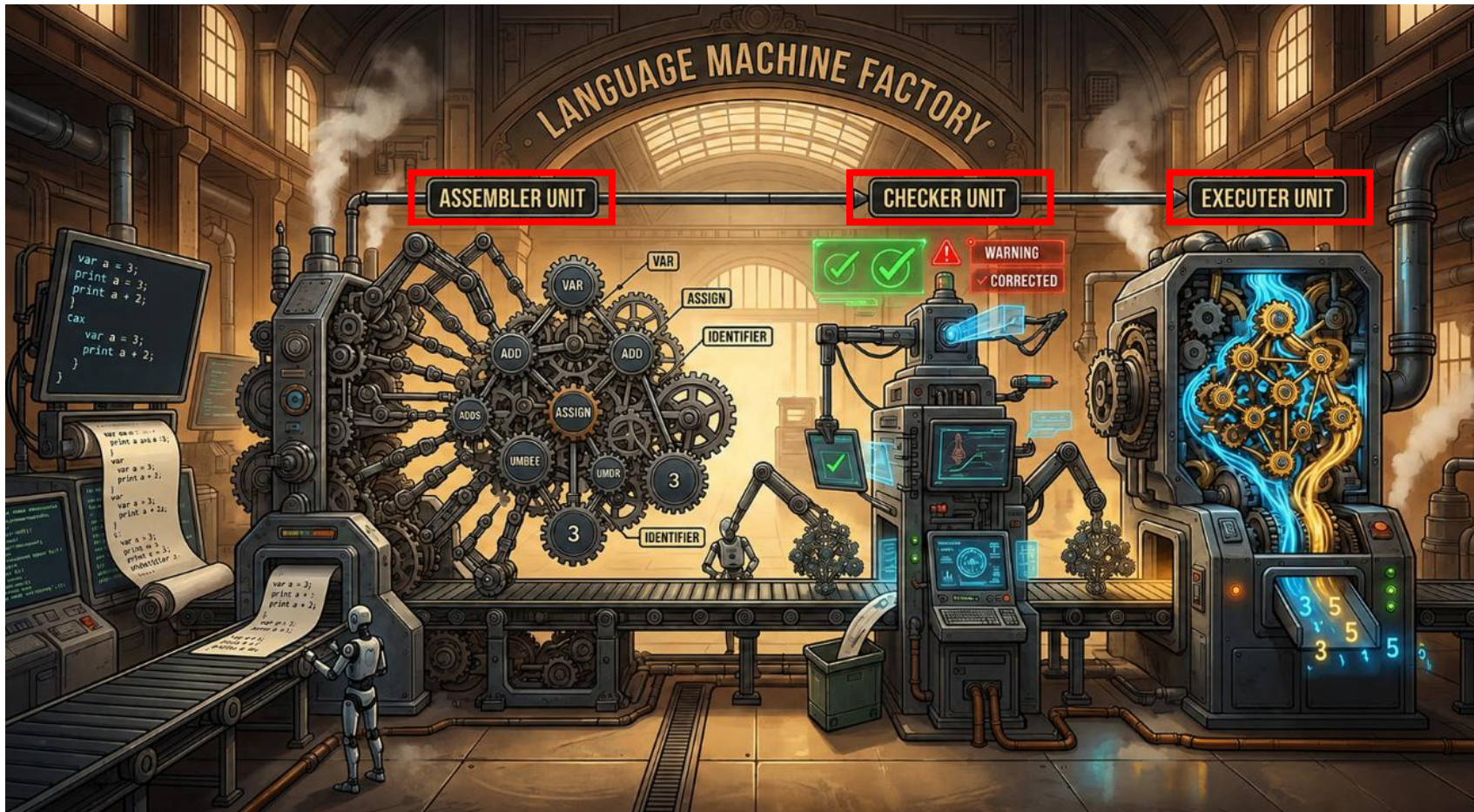
NO!!!



Code Fab 의 구성요소

Code Fab 는 마치 파이프라인을 갖춘 공장처럼 동작되는 인터프리터

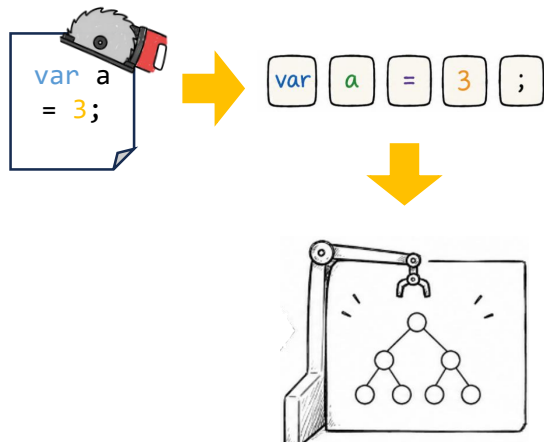
- 총 3개의 Unit 으로 구성되어 있음



프로젝트 구성

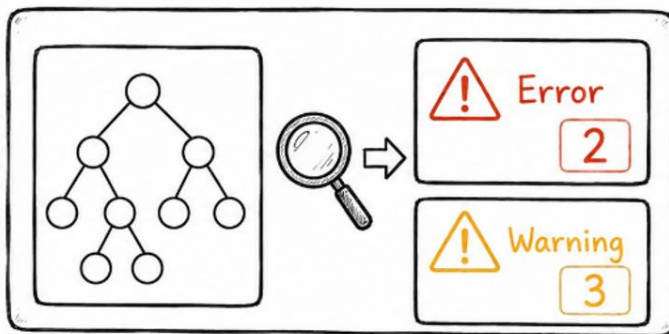
프로젝트는 3개의 Unit 으로 파이프라인이 구성된다.

Assembler Unit



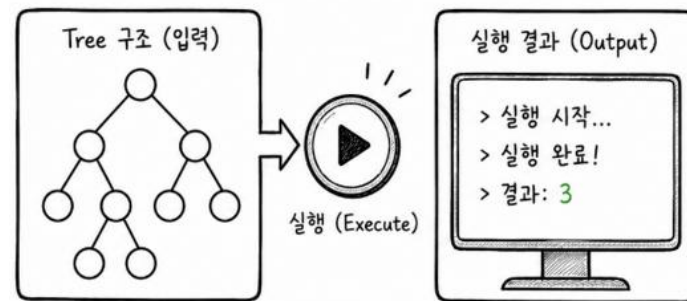
소스코드를 입력받아
부품 (Token) 들로 분해한 뒤,
가공하여
Tree구조 조립체를 만든다

Checker Unit



조립체를 실행하기 전에 오류
및 경고를 검출한다.

Executor Unit



조립체를 실제로 실행하여,
실행 결과가 나온다.

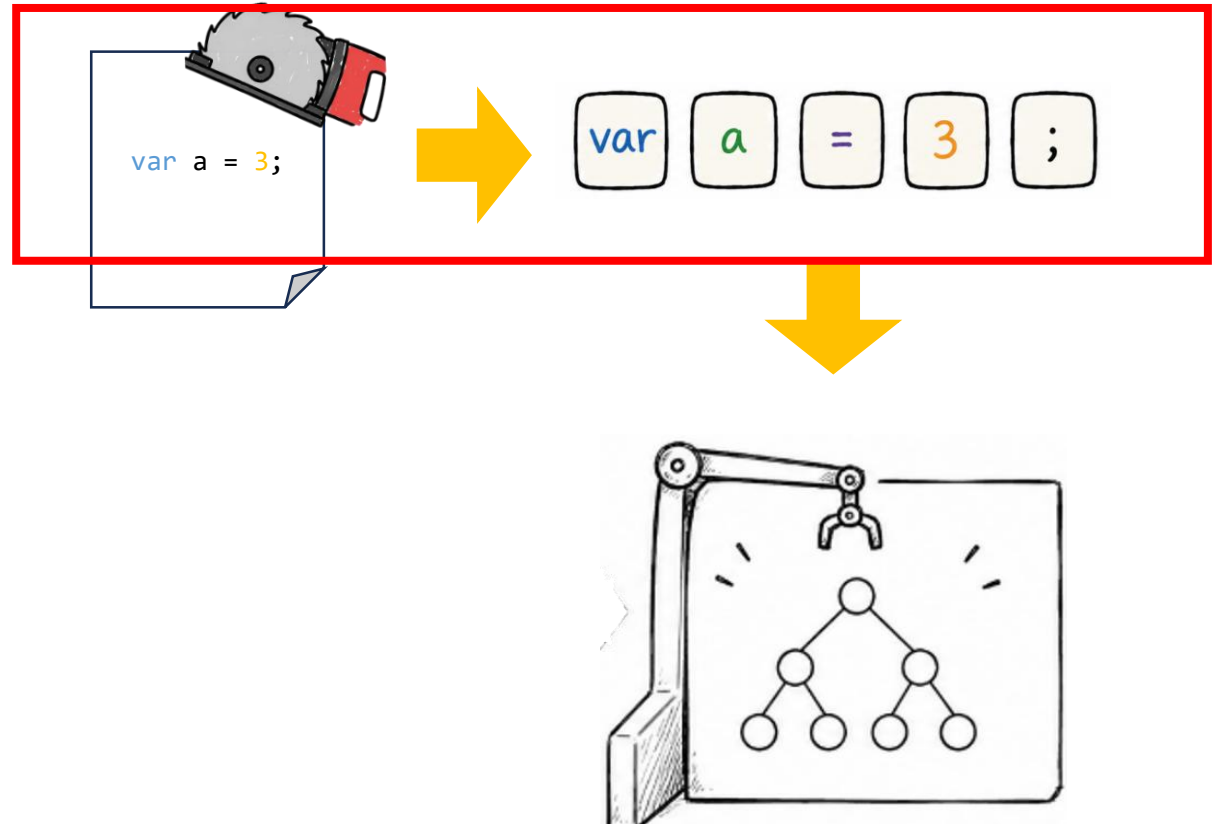
Assembly Unit 핵심원리 - Token화

Token (부품으로 비유)

- 소스코드를 의미 있는 단위로 잘라낸 조각

Token의 종류

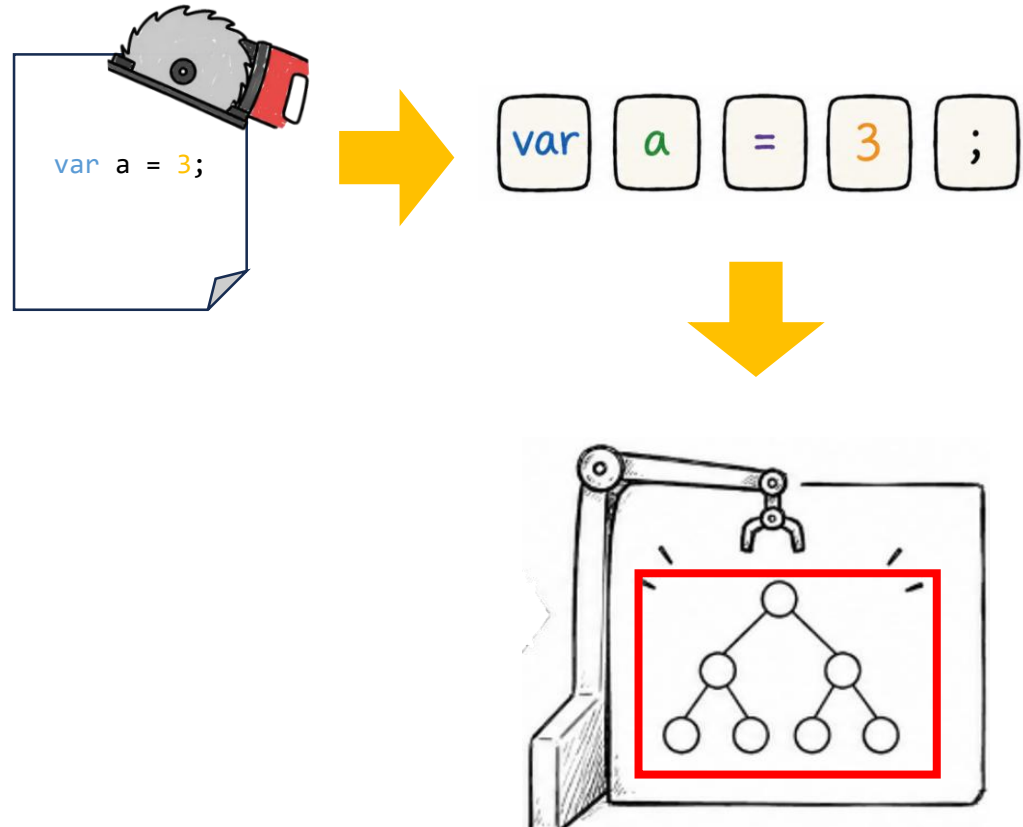
- 키워드
예시 : if, for, return
- 식별자 (Identifier)
예시 : 변수이름, 함수이름
- 리터럴
숫자, 문자열
- 연산자
+ - * / 등
- 구분자 (문장 끝 or 블록 구분)
; } { 괄호 등



Assembly Unit 핵심원리 – 문법 Tree

문법 Tree (Tree구조 조립체로 비유)

- Token 정보를 가공해서, 노드들을 만든다.
이후 노드들을 조립하여 "문법 Tree"를 만든다.
- 이 문법 Tree가 완성되어야만
실행가능한 구조가 된다.



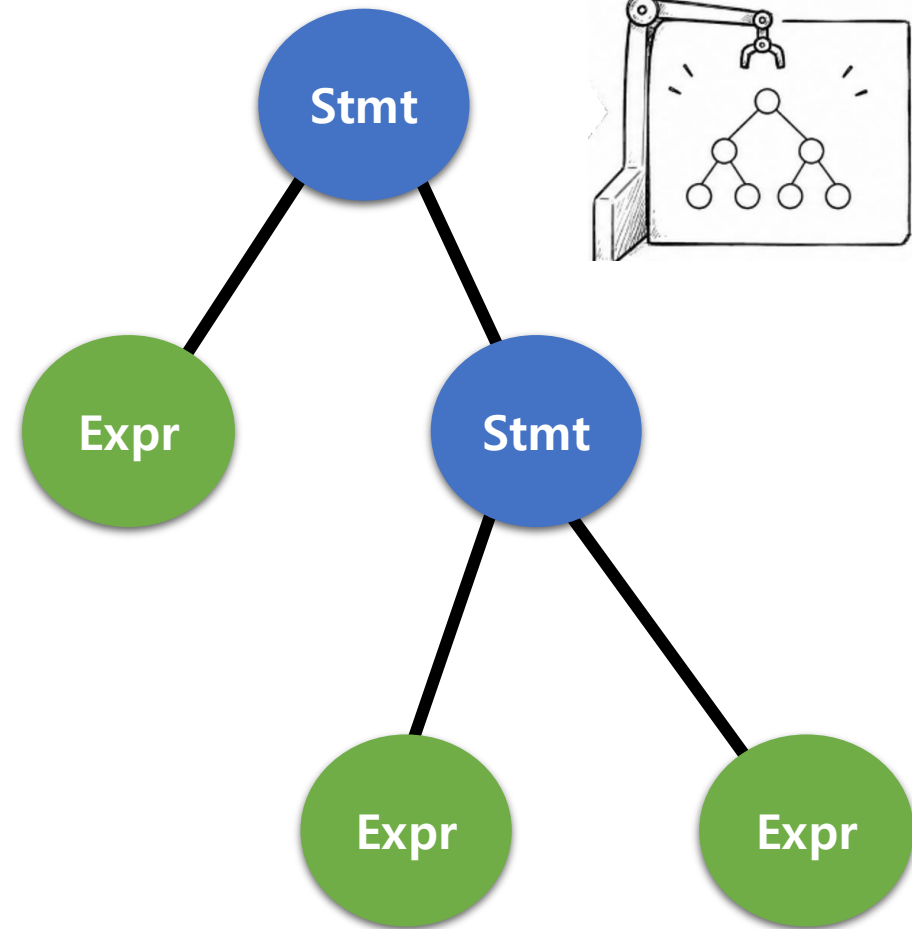
Assembly Unit 핵심원리 – 문법 Tree 의 구성요소

Expression 노드(표현식, 이하 Expr)

- 값을 만드는 것
- 실행하면, 값 하나로 **평가**(치환)되는 코드 단위
 - 예시 : Binary Expression(이항연산 +)
 - 예시 : Assign Expression(대입연산 =)

Statement 노드 (**문장**, 이하 Stmt)

- 값 반환 없이 동작을 수행하는 코드 단위
- 예시
 - 선언**문** – var a (변수 선언)
 - 조건**문**, 반복**문** – if, for

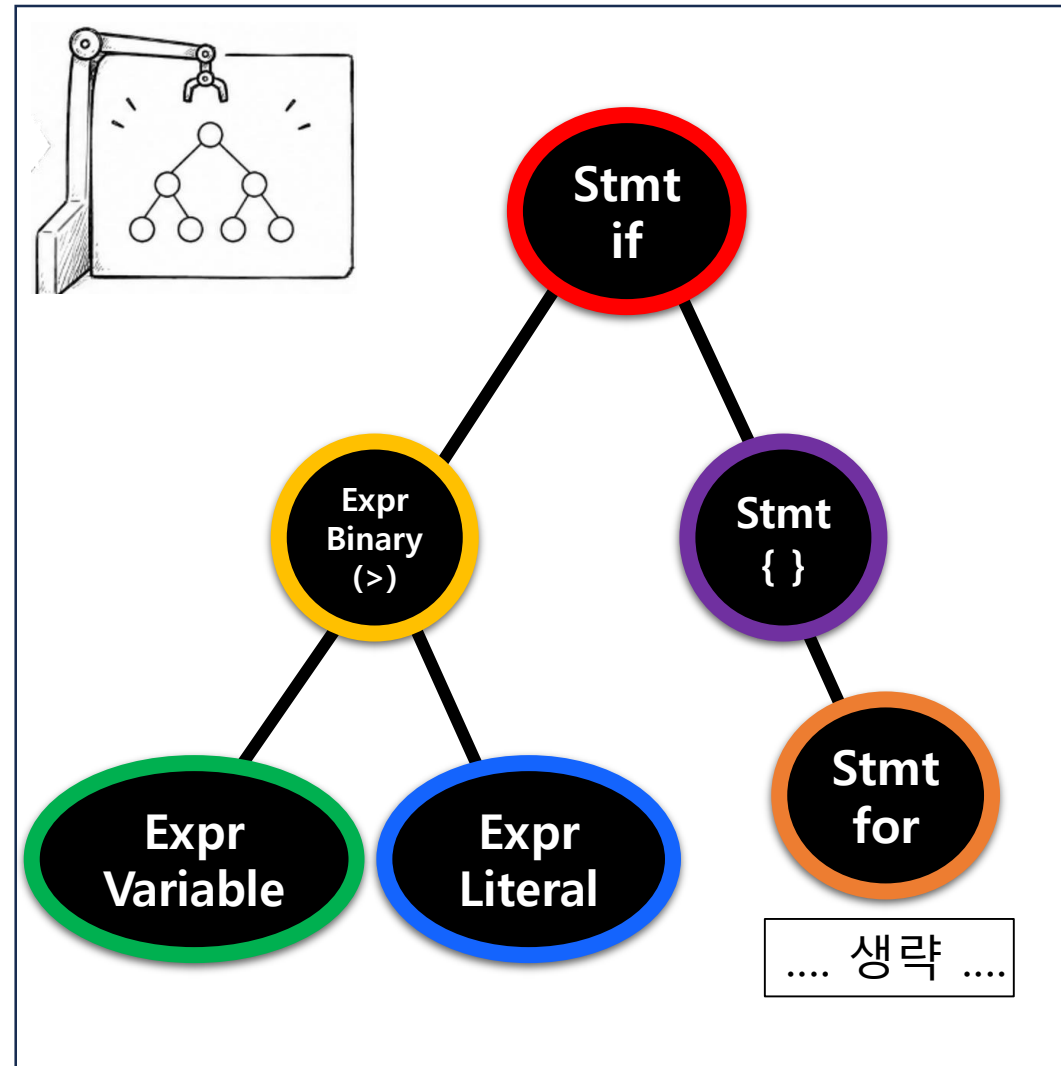


Assembly Unit 핵심원리 – 문법 Tree의 예시

Stmt 노드와 Expr 노드 예시

- 색깔별로 코드 – 노드 대응

```
if(a > 3)
{
    for(...) { ... }
}
```



Checker Unit 핵심원리 – 오류 검사

코드간 규칙들이 잘 지켜졌는지 검사

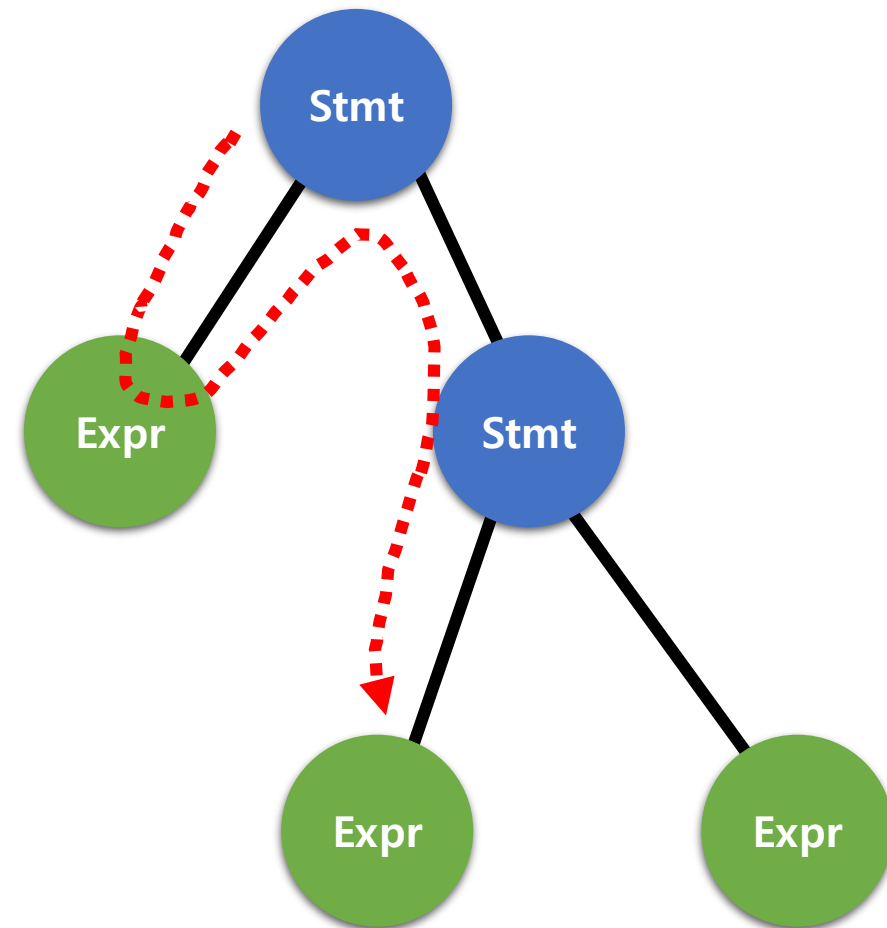
재귀호출로 구현한 DFS 알고리즘을 사용해 각 노드들 탐색

검사하는 것

DFS를 돌면서 각 Stmt, Expr 들이 의미적으로 올바른지 검사

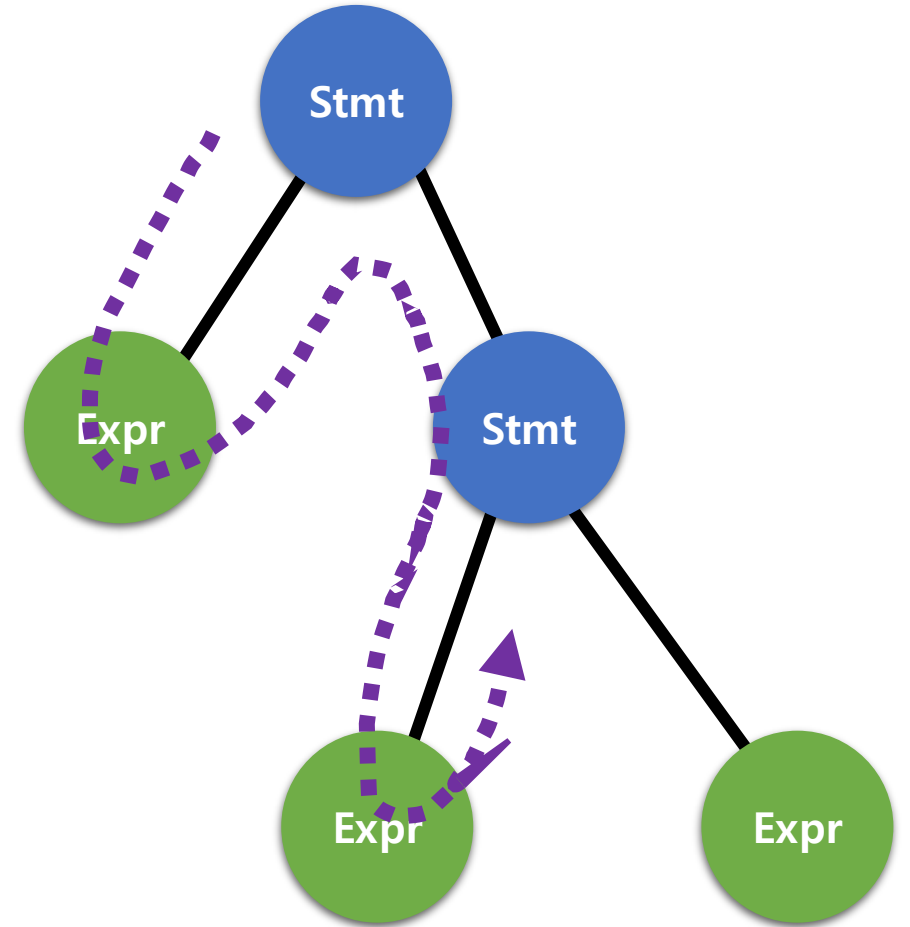
- 변수 중복 선언인지 (같은 블록 내 중복)
- 선언 시 자기 참조

등 코드간의 의미적인 규칙을 어겼는지 검사합니다.



Executor Unit 핵심원리 - 실행하기

문법 Tree가 다 만들어졌으니,
재귀호출으로 구현 된 DFS 알고리즘을 사용하여
각 **Expr**와 **Stmt**를 실행합니다.



Prompt Shell 동작 원리

한줄 한줄 입력 받을 때 마다
3 단계 파이프라인이 모두 수행된다.

```
C:\Users\jeong>python
Python 3.13.7 (tags/v
Type "help", "copyrig
>>>
>>> a = 5
>>> b = 10
>>> print(a + b)
15
>>>
```

Assembler Unit



Checker Unit

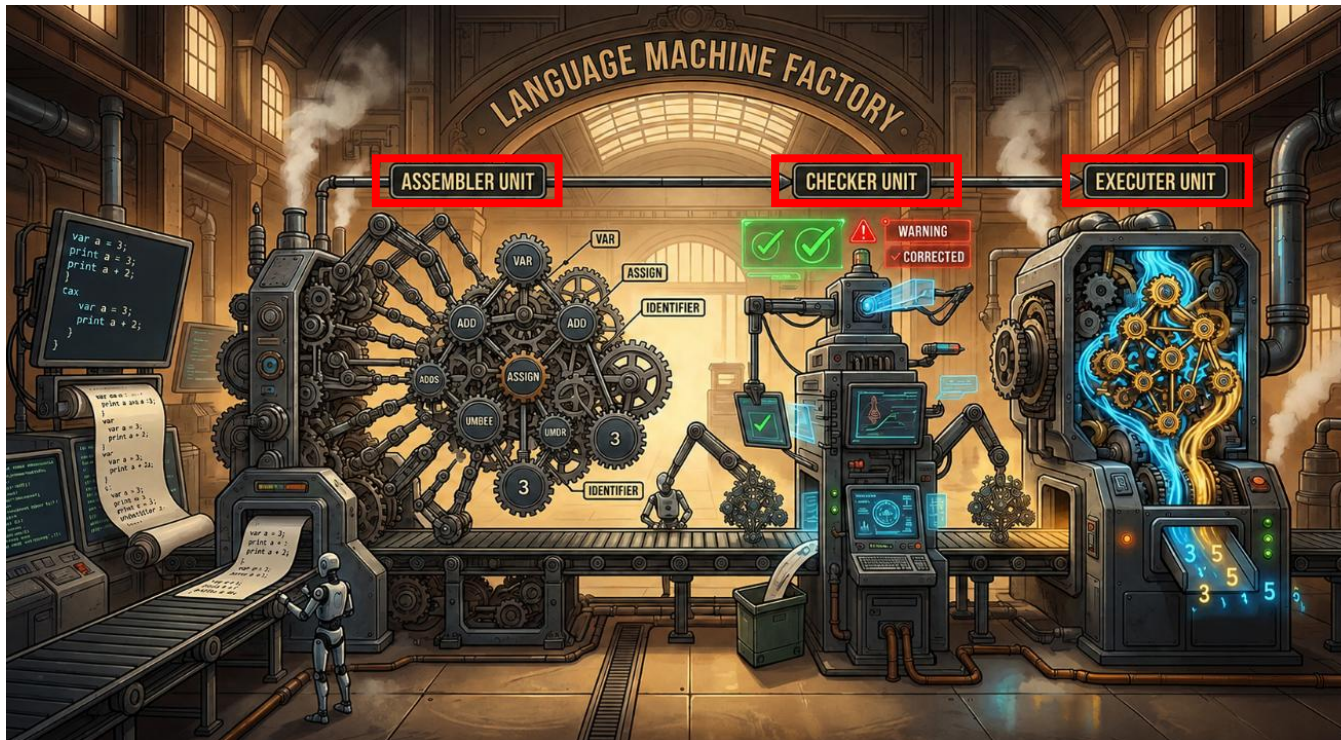


Executor Unit

[정리] Code Fab 동작

Code Fab 은 마치 공장처럼, 한 줄의 프롬프트 입력을 받고,
즉시 세 Unit 공정 (Assembler, Check, Executer) 를 거쳐 실행하는 하나의 공장으로 비유 될 수 있습니다.

이제 여러분께서는 팀만의 프로그래밍 언어를 만들고,
Interpreter 와 PromptShell을 직접 제작해야는 프로젝트를 진행 할 것입니다.



```
C:\Users\jeong>python
Python 3.13.7 (tags/v
Type "help", "copyrig
>>>
>>> a = 5
>>> b = 10
>>> print(a + b)
15
>>>
```

Chapter 02 - 1

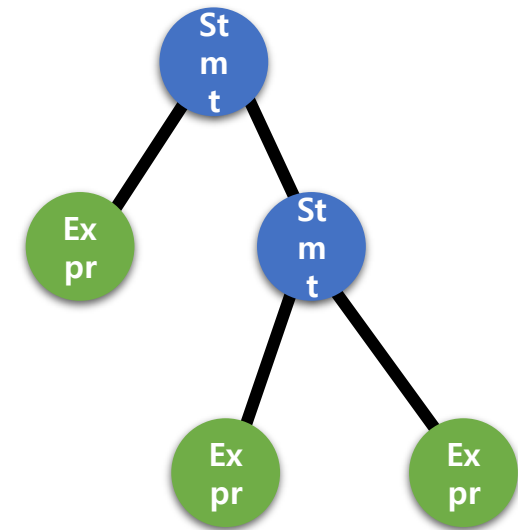
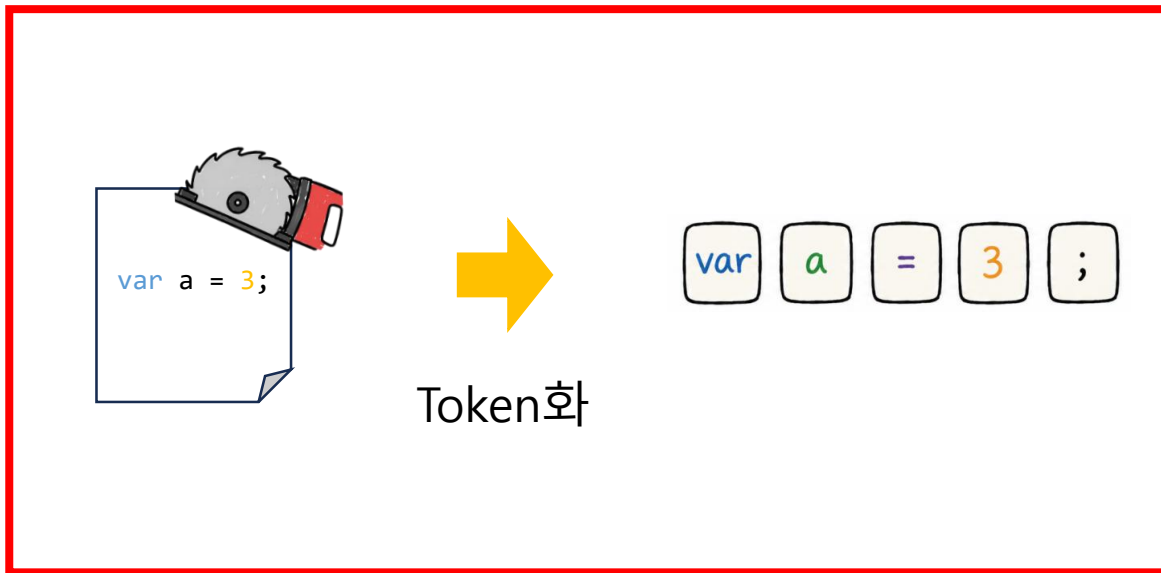
Assembler Unit - Tokenizer

소스코드를 재료로 부품을 생산하고, 가공하여, 최종적으로 Tree 조립체를 만든다.

Assembler Unit

Token 생산과 Expr / Stmt 조립

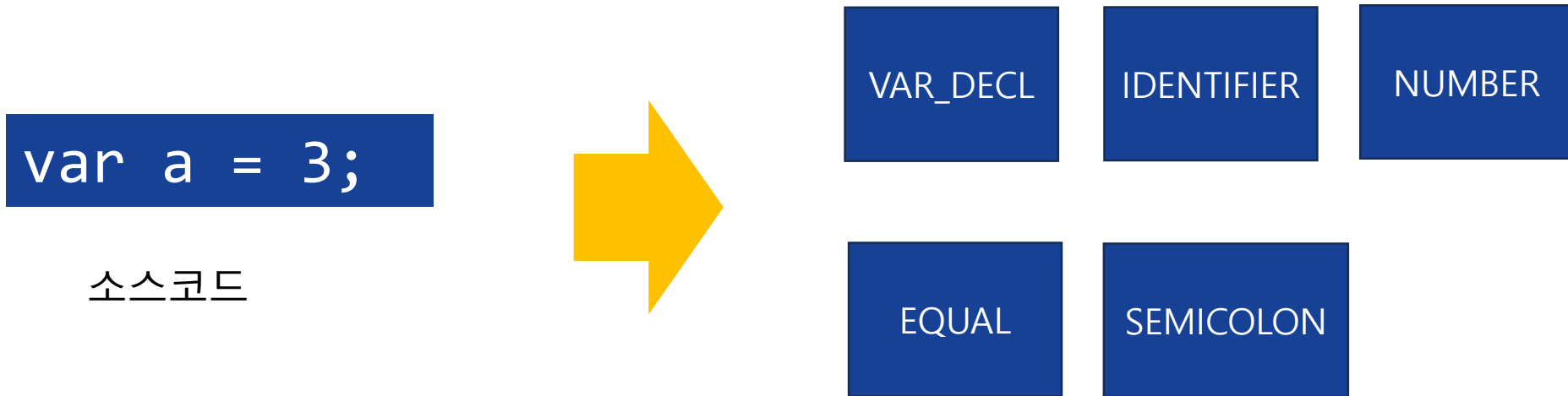
- 1) Assembler Unit 은 프롬프트 셸에서 사용자 입력으로 소스코드를 받는다.
- 2) 소스코드로부터 부품(Token)들을 생산한다.
- 3) 생산된 부품(Token)들을 가공하여 실행할 수 있는 Expr / Stmt 로 만든다.



부품 생산

주어진 소스코드를 여러 Token 으로 분해한다.

- 소스코드를 읽어 단순한 텍스트가 아닌, 의미 있는 최소 단위로 분해한다.



조립체에 사용할 Token List 를 생산한다

생산할 부품(Token) 예시

규격이 정해져 있는 Token

- 키워드 : if , for, var 등
- 연산자 : > <, =, +, -, *, /, !, and, or 등
- 리터럴 : True, False, 숫자 리터럴, 문자열 리터럴 등

그 외

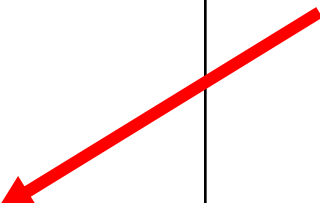
- identifier (식별자), 세미콜론, 소, 중괄호 등

생산할 부품(Token) 예시

Token 을 클래스로 구현할 수 있다.

```
class Token{  
    TokenType type;  
    String origin;  
    ...  
    Token(TokenType type, String origin , ... ) { }  
    ...  
}
```

필요시 다른 정보들을 추가 가능



Token 클래스 예시

→ TokenType 을 필드로 가지고 있고,
원본 문자열을 따로 기록해둔다

Tokenizer 예시 1

입력 : age = 37

출력 : [

Token(IDENTIFIER, "age"), Token(EQUAL, "="), Token(NUMBER, "37", value=37.0), Token(EOF, "")
]

- 1 단계 : 문자 읽기 (예 : "age" 읽기)
- 2 단계 : 문자 패턴 인식 (예 : age 를 identifier 로 인식)
- 3 단계 : 토큰화 (예 : Token(IDENTIFIER, **필요한 정보 추가입력**) 생성)
- 토큰 반복 수집

Tokenizer 예시 2

입력 : if (x > 10)

출력 : [

Token(IF, "if"), Token(LEFT_PAREN, "("), Token(IDENTIFIER, "x"), Token(GREATER, ">"),

Token(NUMBER, "10", value=10.0), Token(RIGHT_PAREN, ")"), Token(EOF, "")

]

Tokenizer 예시 3

입력 : a + b * 3

출력 : [

Token(IDENTIFIER, "a"), Token(PLUS, "+"), Token(IDENTIFIER, "b"), Token(STAR, "*"),

Token(NUMBER, "3", value=3.0), Token(EOF, "")

]

구현해야할 Token 예시 정리 -1

Token Type	예시	비고
LEFT_PAREN, RIGHT_PAREN	()	그룹핑
LEFT_BRACE, RIGHT_BRACE	{ }	블록 스코프
SEMICOLON	;	구분자
PLUS/ MINUS/ STAR/ SLASH	+ - * /	산술연산
EQUAL	=	할당
GREATER/ LESS	> <	비교

구현해야할 Token 예시 정리 - 2

Token Type	예시	비고
IDENTIFIER	x, y, z, calcSum	식별자
STRING	"hello"	문자열
NUMBER	7 3.141592	숫자 (double)
VAR	var	변수 선언
IF / ELSE	if else	조건문
FOR	for	반복문
TRUE / FALSE	true false	불리언
AND / OR	and or	논리
PRINT	print	출력문
EOF		토큰 스트림 끝

이외에 필요한 토큰이 있는 경우 추가로 구현 가능

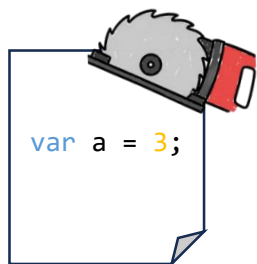
Chapter 02 - 2

Assembler Unit - Expression

소스코드를 재료로 부품을 생산하고, 가공하여, 최종적으로 Tree 조립체를 만든다.

Token 생산과 Expr / Stmt 조립

- 1) Assembler Unit 은 텍스트 or 사용자 입력으로 된 소스코드를 입력받는다.
- 2) 소스코드로부터 부품(Token)들을 생산한다.
- 3) 생산된 부품(Token)들을 가공하여 실행할 수 있는 Expr / Stmt 로 만든다.

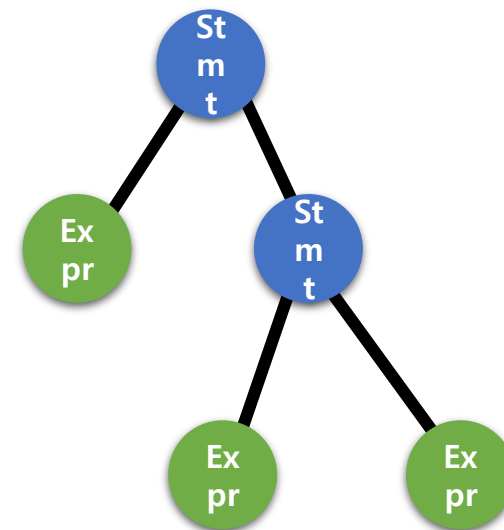


Token화

var a = 3 ;

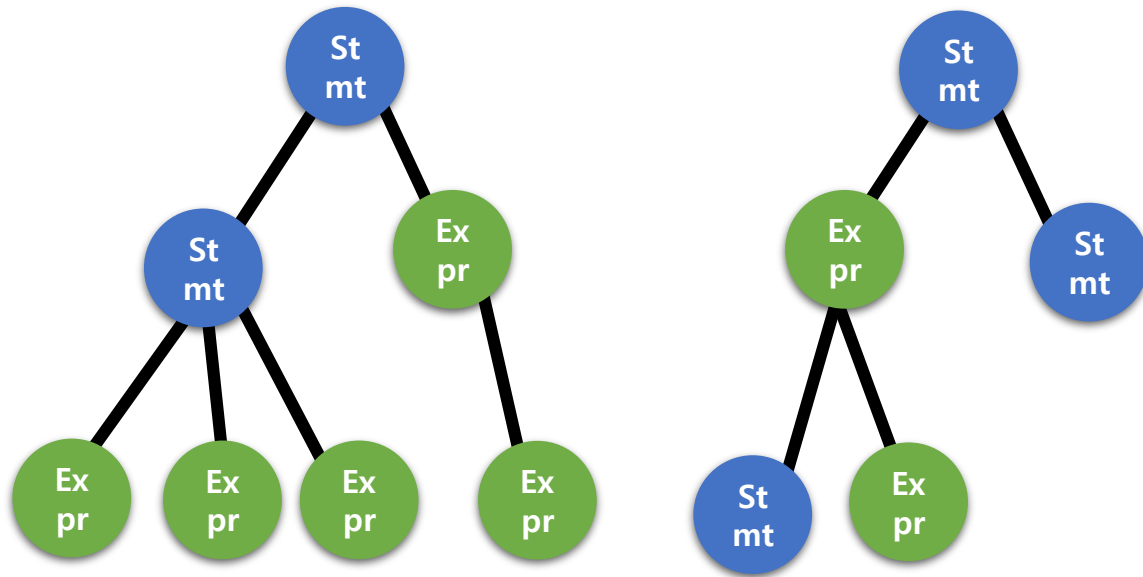


가공

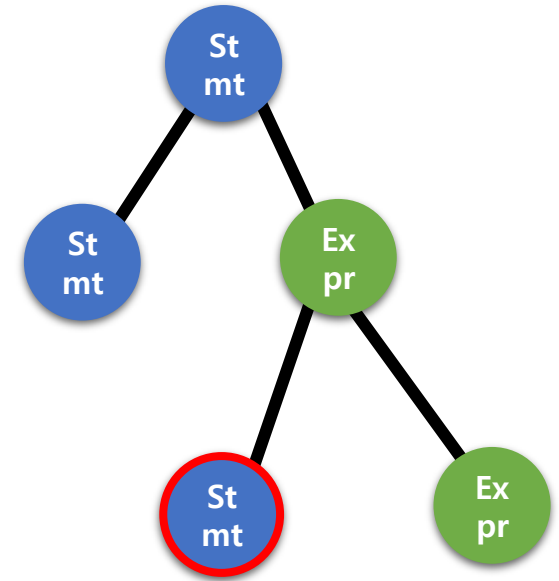


문법 Tree 특징

- Expr는 여러개의 Expr과 Token을 조합하여 만들어진다.
- Stmt는 여러개의 Expr, Stmt, Token을 조합하여 만들어진다.
- Expr 내부에 Stmt를 Child로 갖는것은 허용하지 않는다.



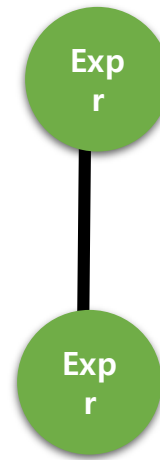
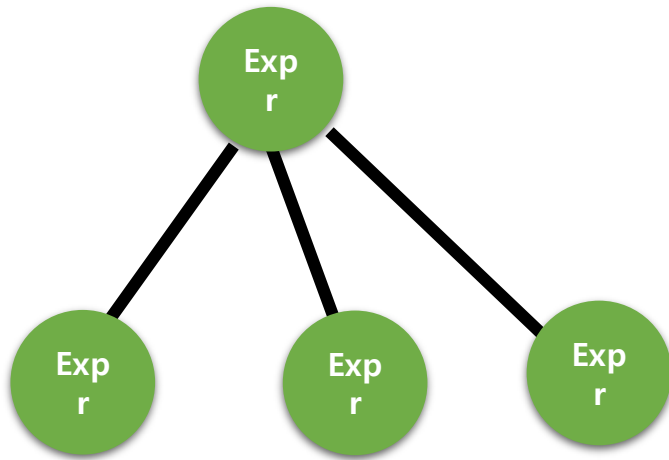
제대로 구성된 문법 Tree 예시



잘못 구성된 문법 Tree 예시

먼저 Expr 부터 살펴본다.

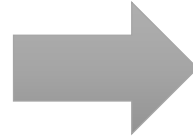
먼저 Expr 을 이해한다. (이후에 Stmt 를 이해해볼 예정)



제대로 구성된 문법 Tree 예시

Child가 없는 단일 구조

>>> 123



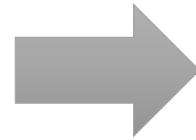
Literal Expr(123)

Expr – Variable Expression

Expr

Variable Expression = 변수의 이름(식별자)을 나타내는 노드

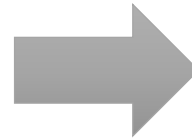
```
>>> a
```



Variable Expr(a)

Child가 없는 단일 구조

>>> true



Boolean
LiteralExpr(true)

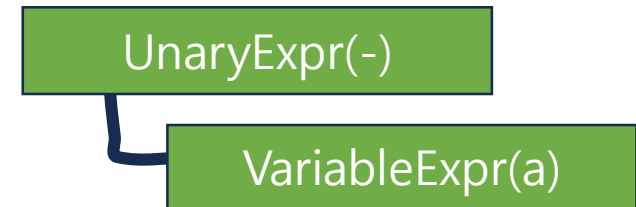
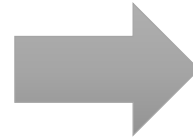
- Literal 하나로 통합도 가능,
- 또는
Boolean Literal, String Literal 등으로
분리 가능

Expr - Unary Expression

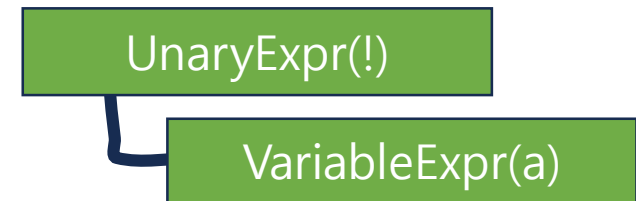
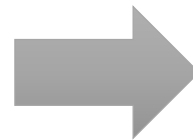
Unary Operator : 값을 1개만 사용하는 연산자

종류 : !, +, -

>>> -a



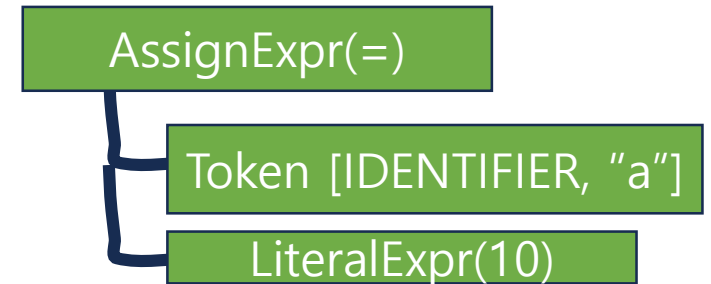
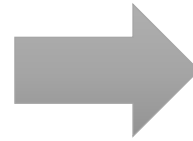
>>> !a



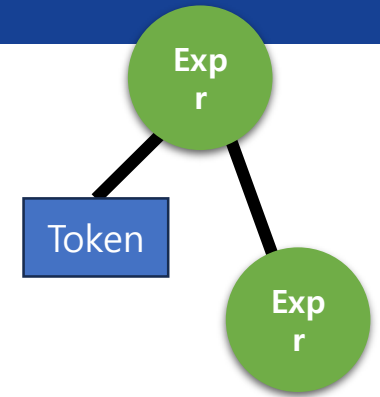
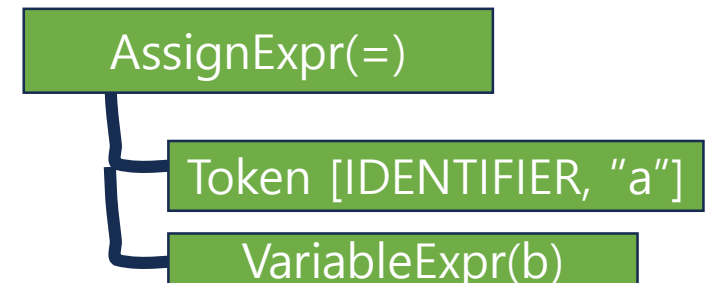
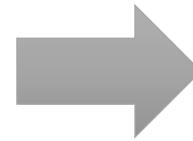
Expr - Assign Expression

Assign Expression : 변수에 값을 대입하는 노드

>>> a=10



>>> a=b

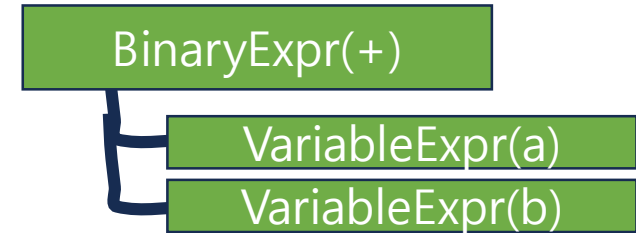
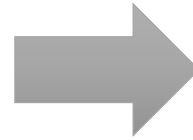


Expr - Binary Expression

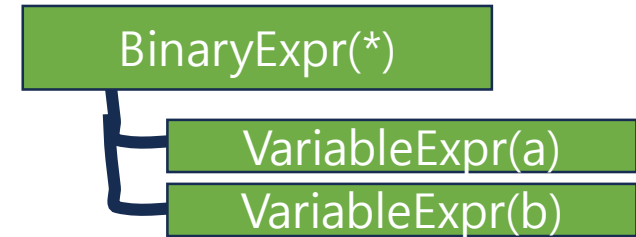
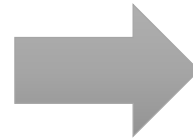
Binary Expression : 값을 2개 사용하는 연산자

종류 : + * >

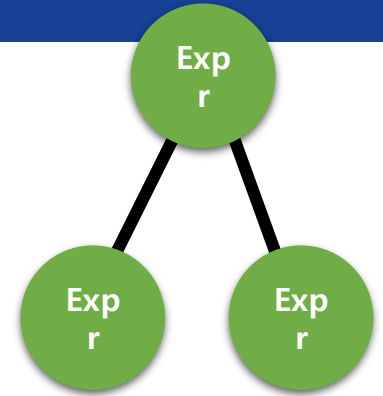
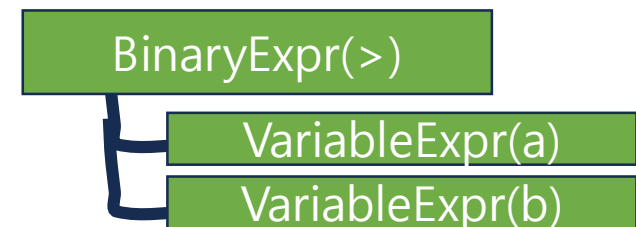
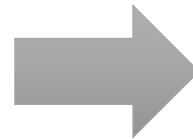
>>> a+b



>>> a*b

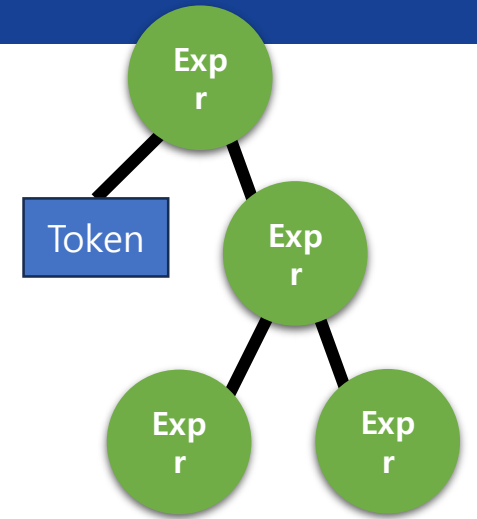


>>> a>b

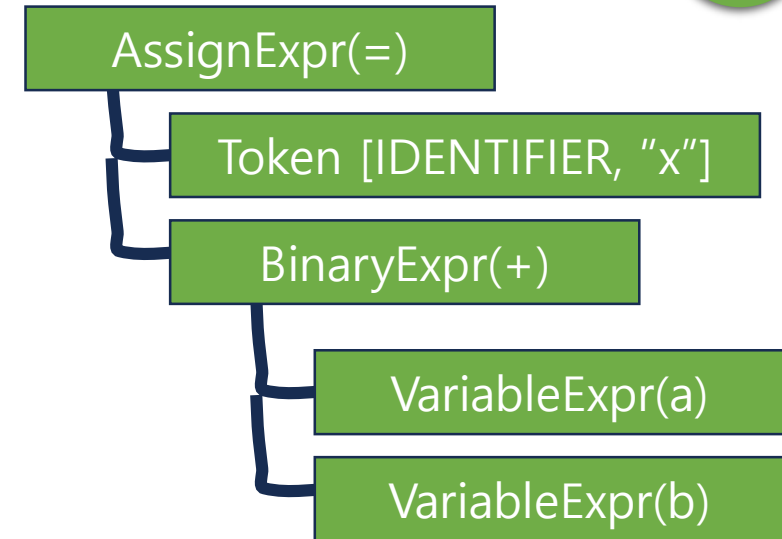
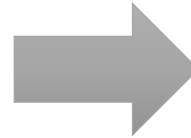


Expr - Assign Expression

Assign Expression : 변수에 값을 대입하는 노드



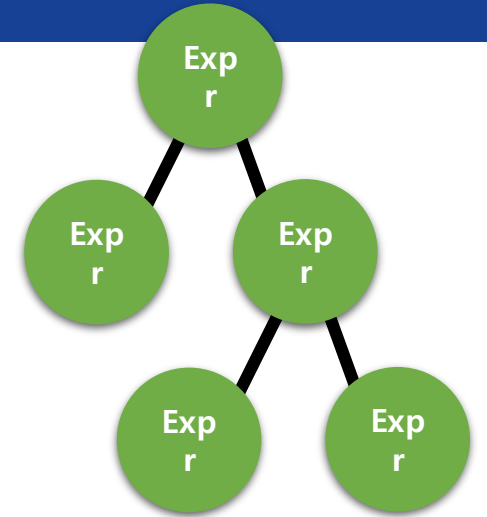
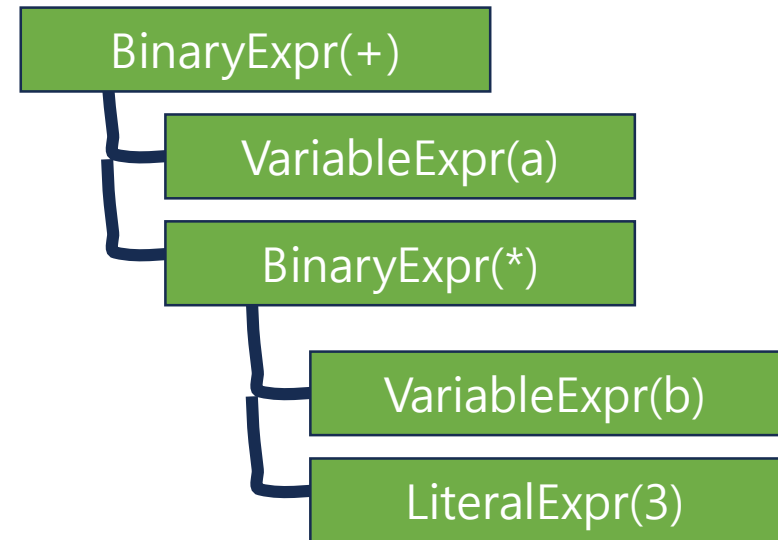
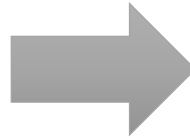
>>> **x = a + b**



Expr - 연산 우선순위

우선순위에 맞추어, 트리가 구성된다.

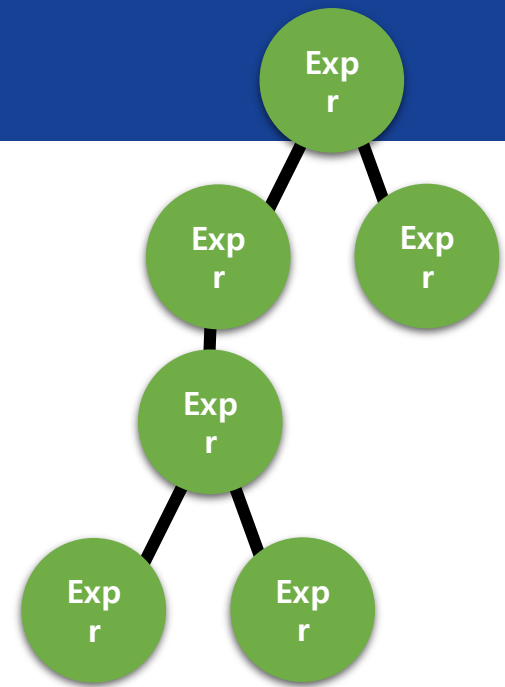
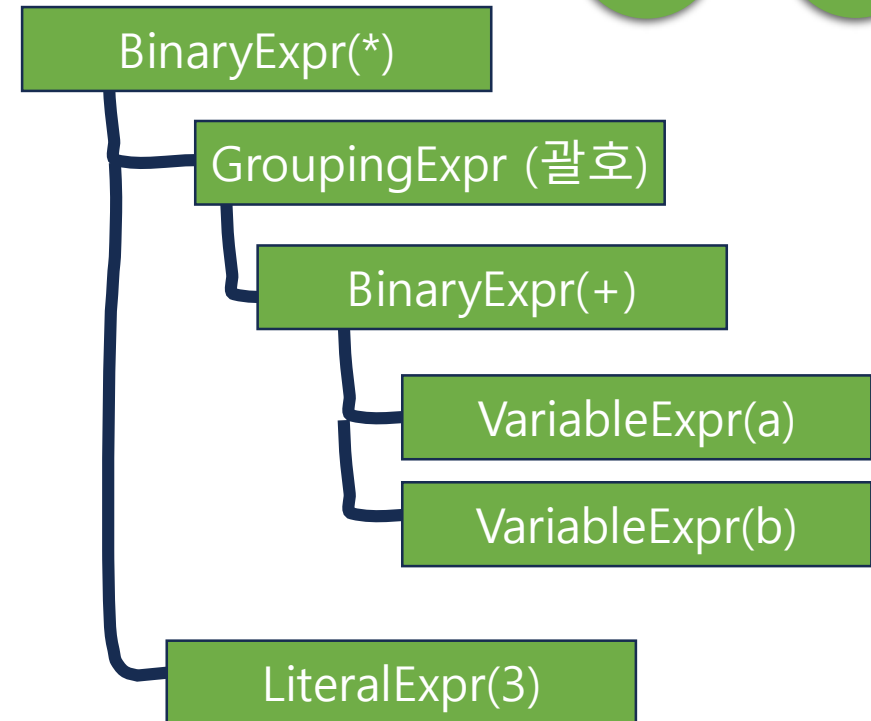
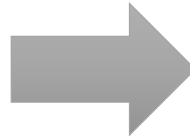
>>> a + b *



Expr - 연산 우선순위

우선순위에 맞추어, 트리가 구성된다.

>>> (a + b) * 3



Expression 의 특징은 실행시 평가(evaluate)가 가능하다는 것이다.

값 생성

- **Literal Expr** : 리터럴 값을 생성
- **Variable Expr** : 변수 참조

연산

- **Binary Expr** : 이항 연산식 (+, -, *, /, < , > 등)
- **Unary Expr** : 단항 연산식 (-, !)
- **Logical Expr** : 논리 연산식 (and, or)
- **Grouping Expr** : 괄호 묶는 연산식 (())

대입

- **Assign Expr** : 변수 대입

구현해야할 Expr 정리

Expr Type	예시
Literal	3, "hi", true
Variable	a (변수)
Assign	a = 3
Binary	1 + 2, a > 0, 2 * 3
Unary	- x, !isExist
Grouping	(1 + 2)
Logical	a and b

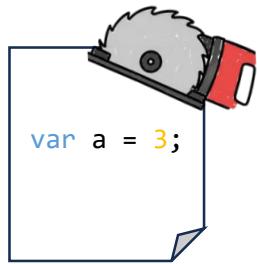
Assembler Unit - Statement

소스코드를 재료로 부품을 생산하고, 가공하여, 최종적으로 Tree 조립체를 만든다.

[복습] Assembler Unit

Token 생산과 Expr / Stmt 조립

- 1) Assembler Unit 은 텍스트 or 사용자 입력으로 된 소스코드를 입력받는다.
- 2) 소스코드로부터 부품(Token)들을 생산한다.
- 3) 생산된 부품(Token)들을 가공하여 실행할 수 있는 Expr / Stmt 로 만든다.

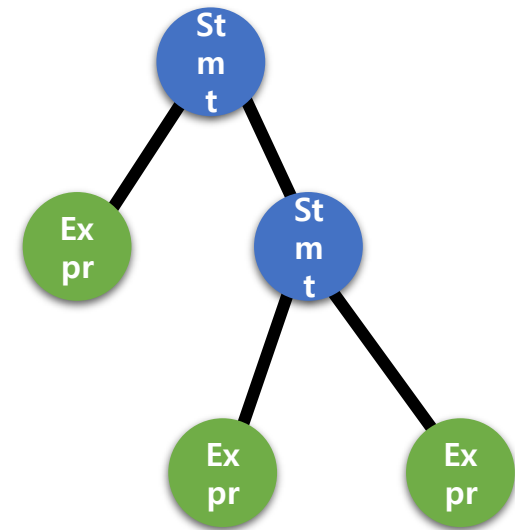


Token화

var a = 3 ;

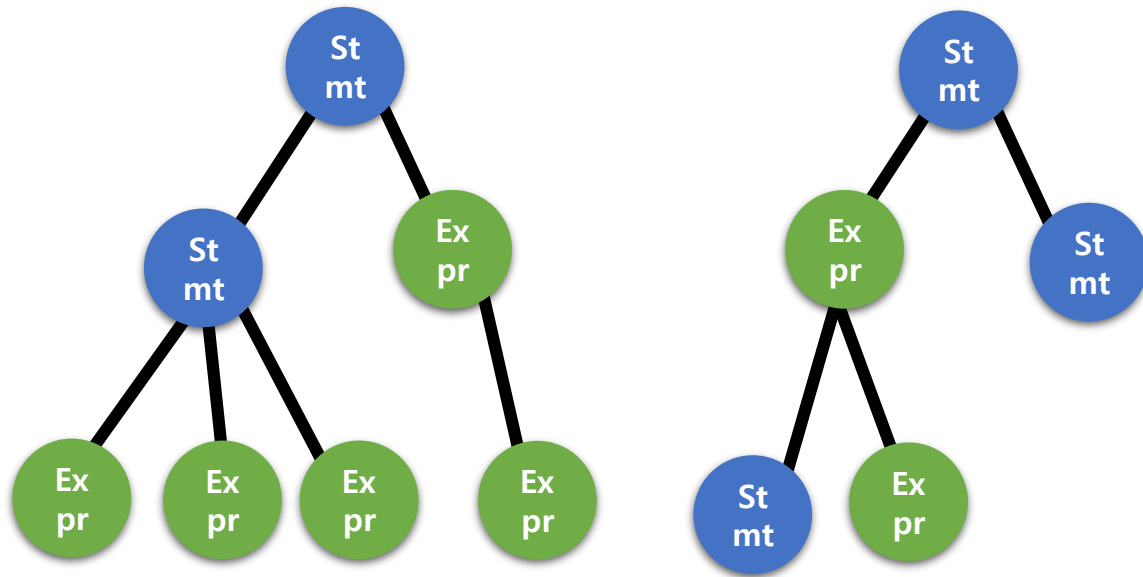


가공

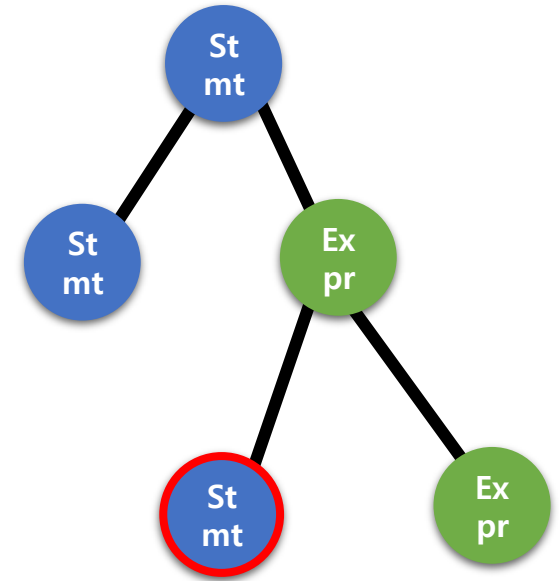


[복습] 문법 Tree 특징

- Expr는 여러개의 Expr과 Token을 조합하여 만들어진다.
- Stmt는 여러개의 Expr, Stmt, Token을 조합하여 만들어진다.
- Expr 내부에 Stmt를 Child로 갖는것은 허용하지 않는다.
- 트리의 루트(최상위)는 항상 Stmt이다.
- Token 은 노드가 아니라 각 노드의 필드로 보관된다.



제대로 구성된 문법 Tree 예시



잘못 구성된 문법 Tree 예시

조립체 생산

Expr은 값을 계산하는 것

Statement는 행동을 수행하는 것이다.

x = 3

Expression
계산 수행

x = 3;

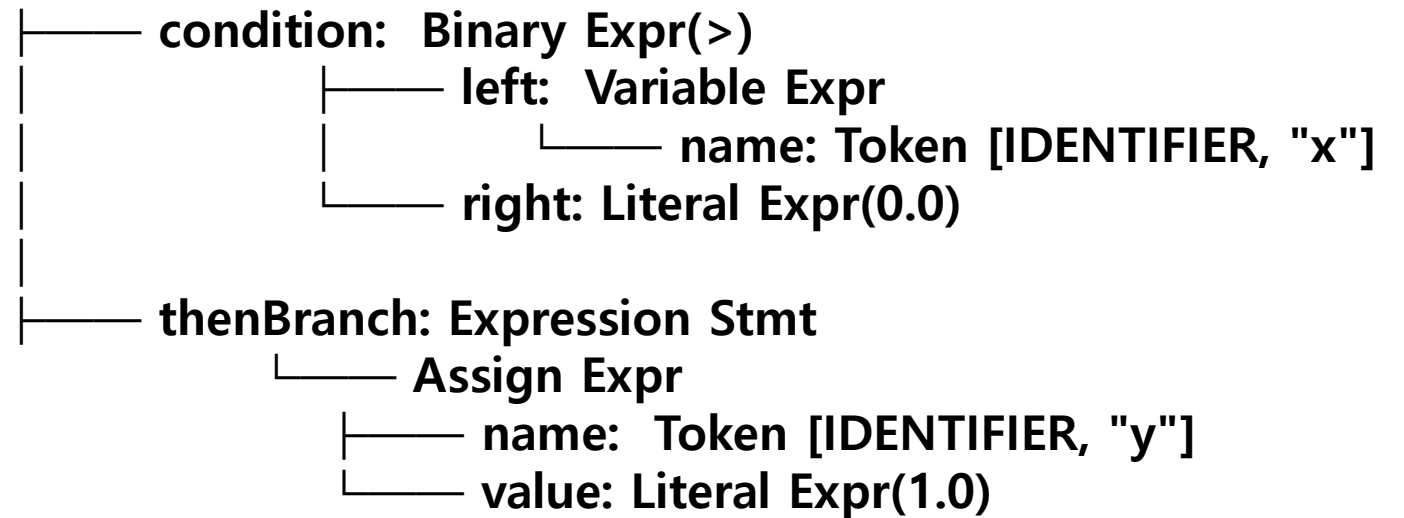
Statement
실행가능한 한 문장

IfStmt

조건에 따라 실행을 분기하는 Statement

```
if (x > 0)
    y = 1;
```

If Stmt



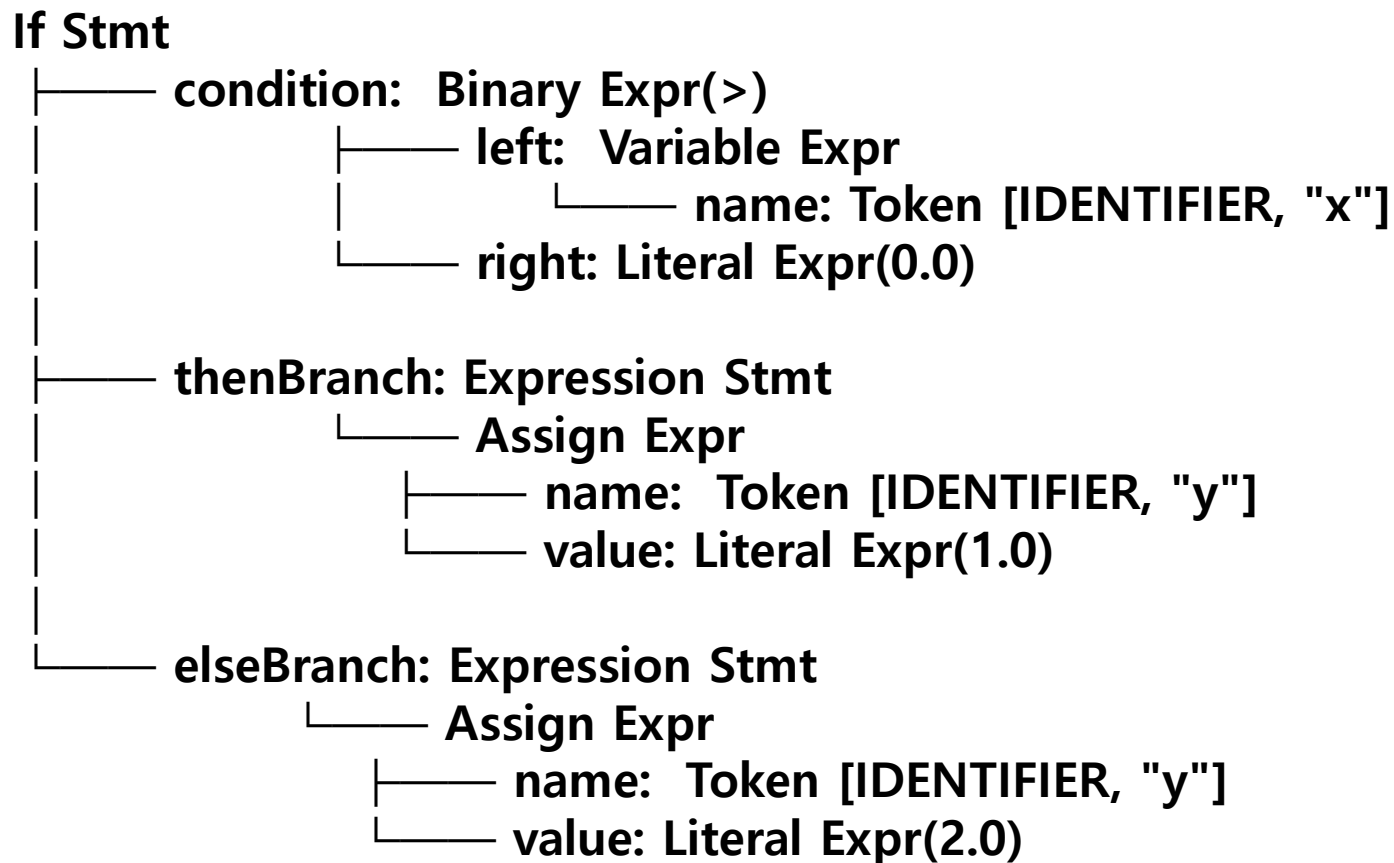
* Expression Stmt : Expr 을 감싸는 Stmt (Wrapper)

IfStmt

조건에 따라 실행을 분기하는 Statement

```
if (x > 0)
    y = 1;
else
    y = 2;
```

* Expression Stmt : Expr 을 감싸는 Stmt (Wrapper)



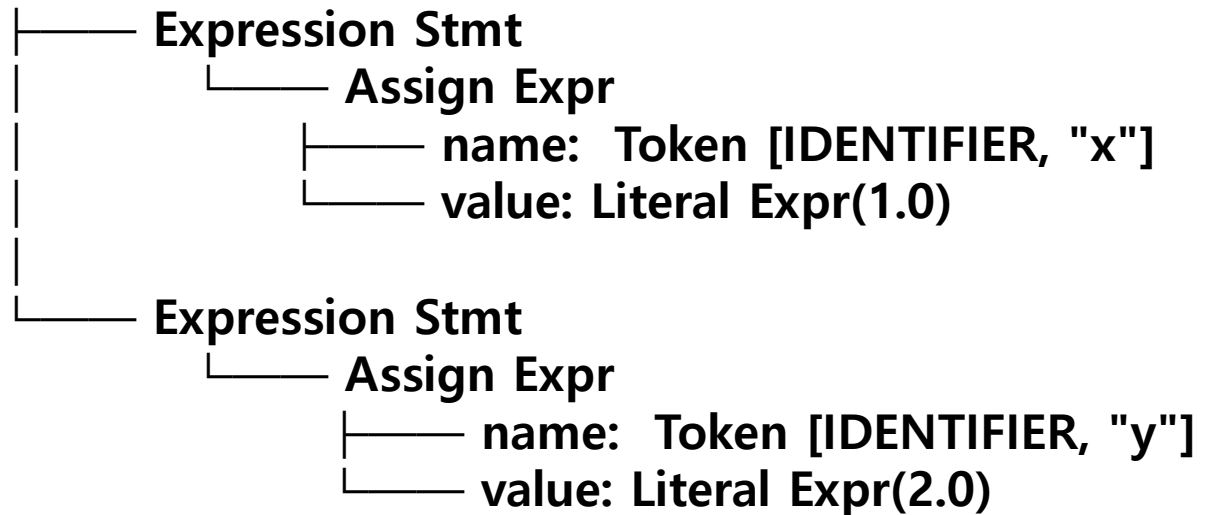
조립체 생산

BlockStmt

Statement 여러 개를 묶는 Statement

```
{  
    x = 1;  
    y = 2;  
}
```

Block Stmt



* Expression Stmt : Expr 을 감싸는 Stmt (Wrapper)

변수 선언(var 개념)

```
var x = 10;
```

VarDeclareStmt

- name : Token [IDENTIFIER, "x"]
- initializer : LiteralExpr(10)

구현해야할 Stmt 정리

Stmt Type	예시
Expression	<code>a + 1;</code> (Expr을 Stmt으로, Warpper)
Print	<code>print a;</code>
VarDeclare	<code>var a = 3;</code>
Block	<code>{ }</code>
If	<code>if (a > 0) { ...} else { ... }</code>
For	<code>for(var i = 0; i < 3; i = i + 1){ ... }</code>

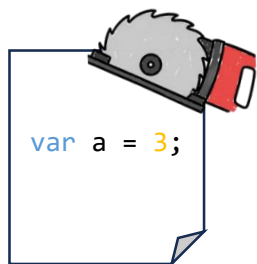
Chapter 02 - 4

Assembler Unit – 가공 작업

소스코드를 재료로 부품을 생산하고, 가공하여, 최종적으로 Tree 조립체를 만든다.

Token 생산과 Expr / Stmt 조립

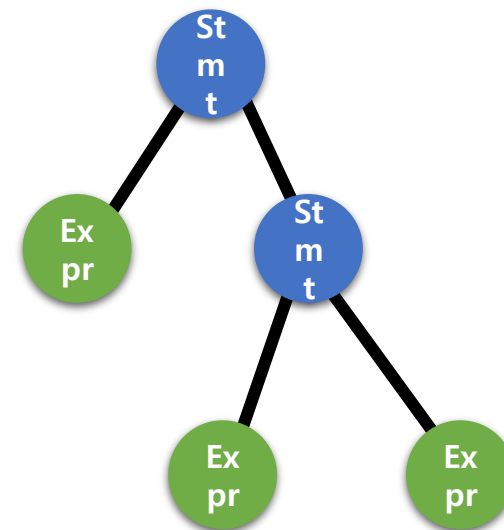
- 1) Assembler Unit 은 텍스트 or 사용자 입력으로 된 소스코드를 입력받는다.
- 2) 소스코드로부터 부품(Token)들을 생산한다.
- 3) 생산된 부품(Token)들을 가공하여 실행할 수 있는 Expr / Stmt 로 만든다.



→
Token화

`var` `a` `=` `3` `;`

→
가공



문법 규칙 $\leftrightarrow$ 문법 트리

문법 트리는 특정 코드의 문법 규칙들이 그대로 적용된 트리이다.

예를들어, if Stmt 의 문법규칙을 살펴보면 아래와 같다.

• **규칙** : if Stmt $\rightarrow$ "if" "(" **expression** ")" **statement** ("else" **statement**)?

? 는 옵션이라는 의미

$\rightarrow$ if 문은 if (expression) statement 로 구성 되어있고 옵션으로 else statement 가 올 수 있다.

- 1) if Stmt 의 토큰들은 위 규칙에 정의된 순서대로 등장 ("if" "(" expression ... 이순서로 등장)
- 2) "if", "(", ")" : 반드시 해당 문자열이 해당 자리에 위치 $\rightarrow$ 다른 토큰으로 대체 불가
- 3) **expression ,statement** : expression 과 statement 는 또 다른 규칙이 적용되는 자리

(3)과 같은 자리에는 다른 규칙들로 확장되기 때문에

문법 트리는 자연스럽게 재귀 구조의 트리가 된다.

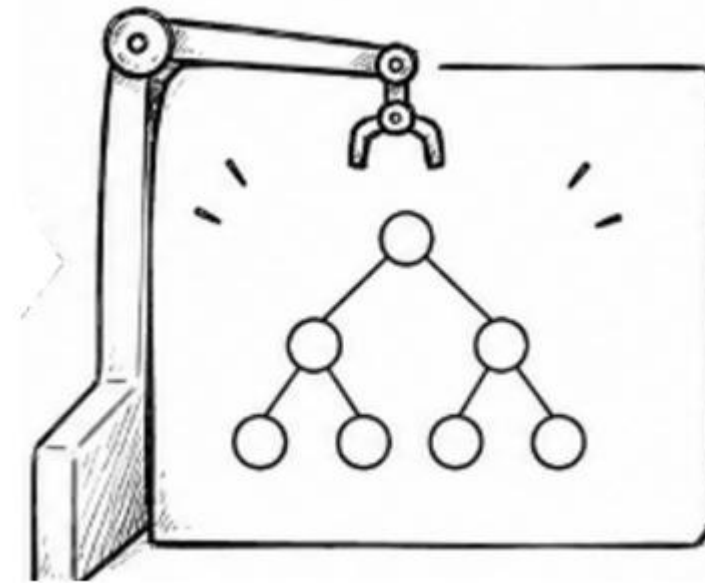
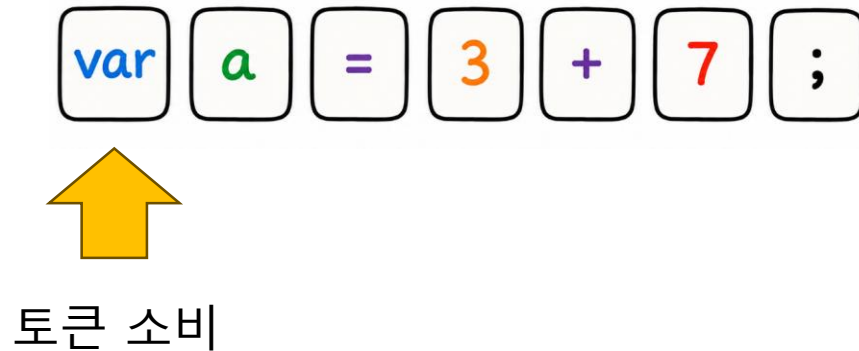
단, 실행에 불필요한 토큰(=, ; 등) 은 노드로 만들지 않는다.

Assembly Unit 핵심원리 – Parser 의 조립과정

문법 규칙에 맞게 Parser 가 동작해야한다.

조립과정

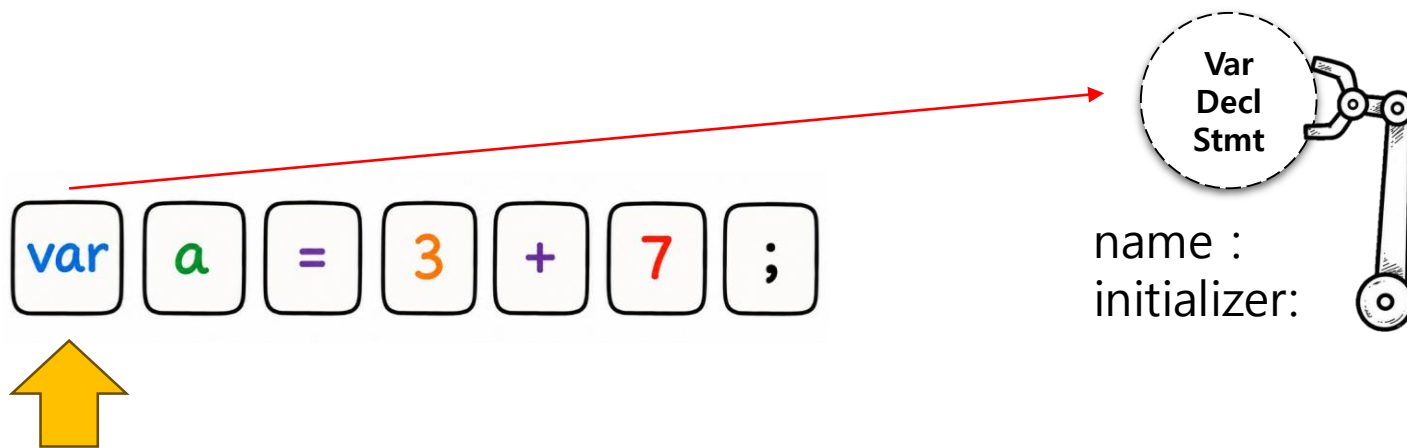
토큰들 하나씩 소비 → 노드 구성 → 트리구조로 조립



Assembly Unit 핵심원리 – 조립과정 1

Var Declare Stmt 노드 생성 준비

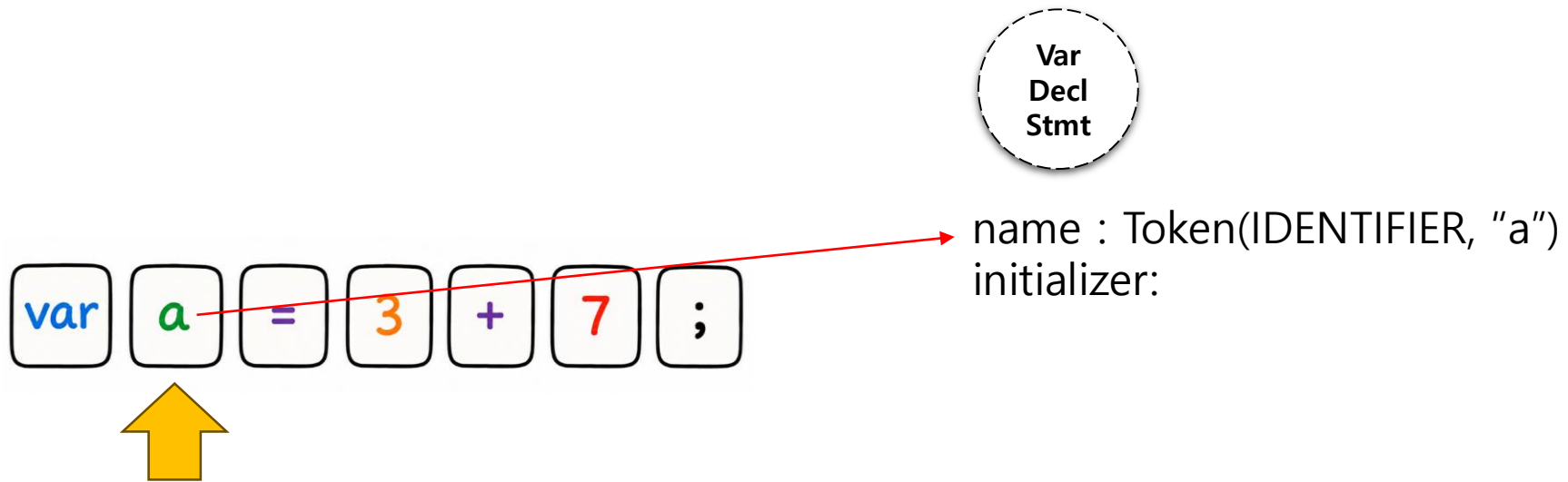
- var 토큰을 소비하여 Var Declare Stmt 을 생성준비한다.
- 노드에는 변수이름(name), 초기화식(initializer) 필드가 존재
- 변수이름이나 초기화식이 아직 미결정이므로 생성만 준비



Assembly Unit 핵심원리 – 조립과정 2

Identifier 토큰 소비

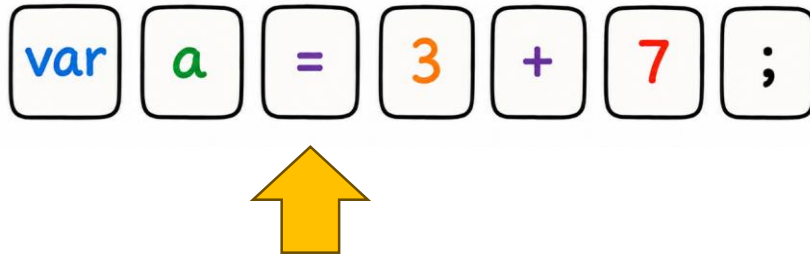
- 변수 이름에 해당하는 Identifier 토큰을 소비하여 name 필드를 채움



Assembly Unit 핵심원리 – 조립과정 3

Equal 토큰 소비

- Equal 토큰은 트리에 넣지는 않는다.

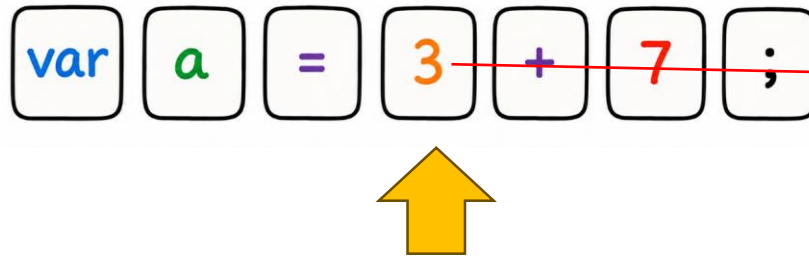


name : Token(IDENTIFIER, "a")
initializer:

Assembly Unit 핵심원리 – 조립과정 4

왼쪽 피연산자 3 조립

- 3을 Literal Expr (3) 으로 생성한다.
- 오른쪽 피연산자가 없을 수도 있으나, 이것은 operator 가 결정되어야 알 수 있는 부분이므로 우선 left 에 넣는다.
→ 만약 operator 없으면 단독 표현식으로 끝



name : Token(IDENTIFIER, "a")

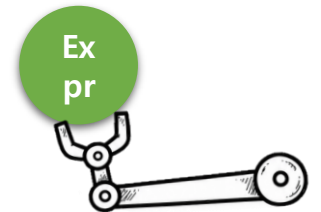
initializer:

left : Literal Expr(3)

op : ???

right : ???

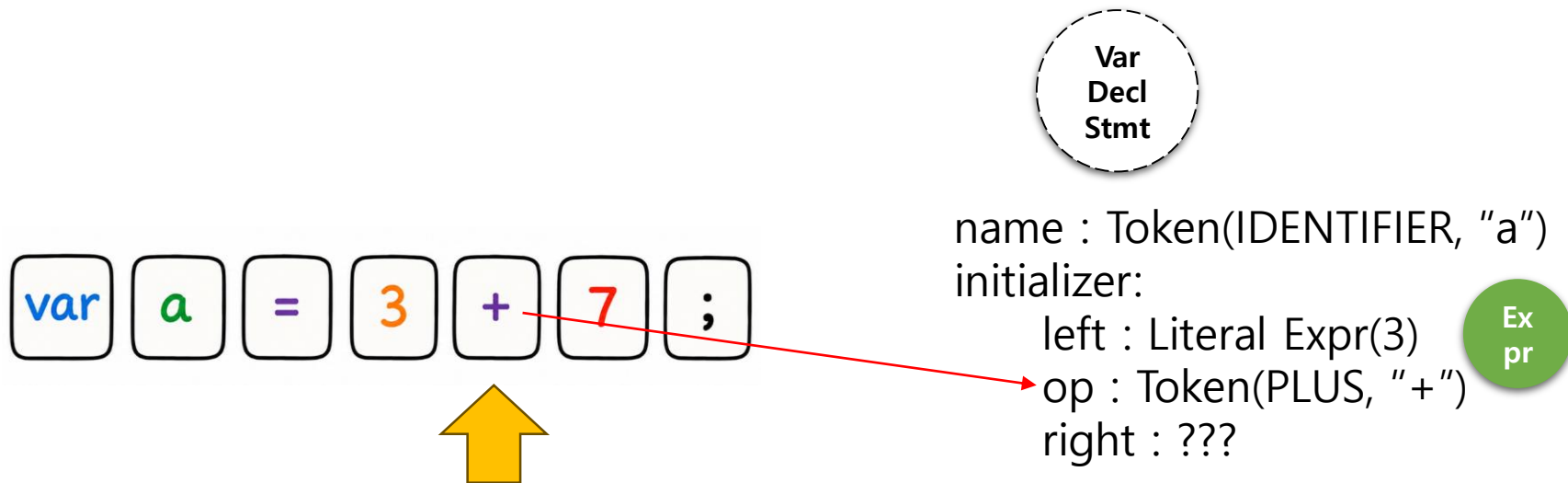
Var
Decl
Stmt



Assembly Unit 핵심원리 – 조립과정 5

+ 발견 → 이항 연산자로 인식

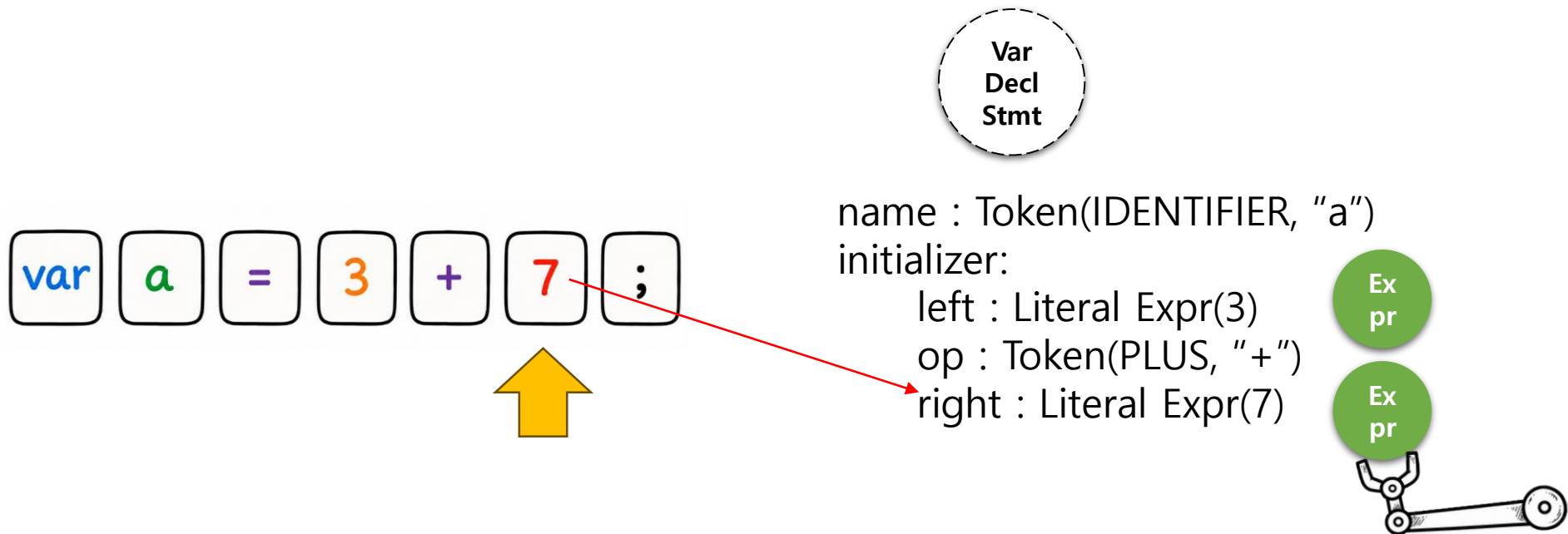
operator 에 PLUS Token을 입력한다.



Assembly Unit 핵심원리 – 조립과정 6

오른쪽 피연산자 조립

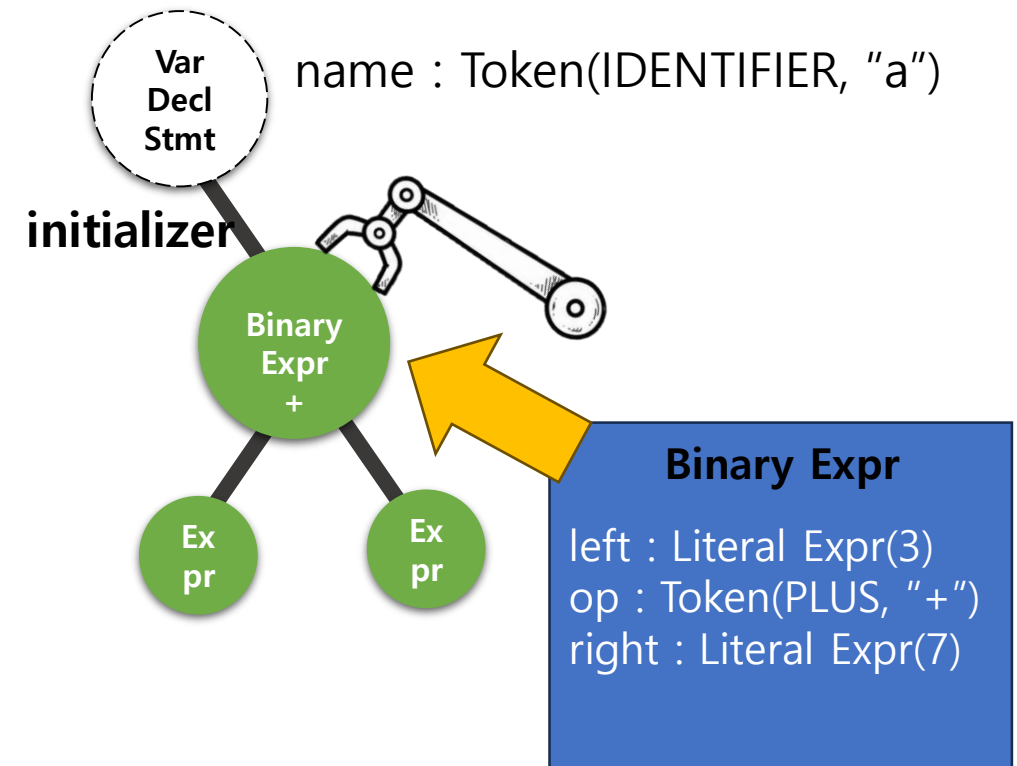
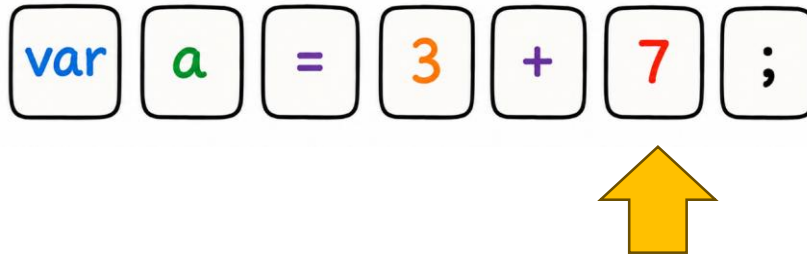
- 7 을 Literal Expr (7) 으로 생성한다.



Assembly Unit 핵심원리 – 조립과정 7

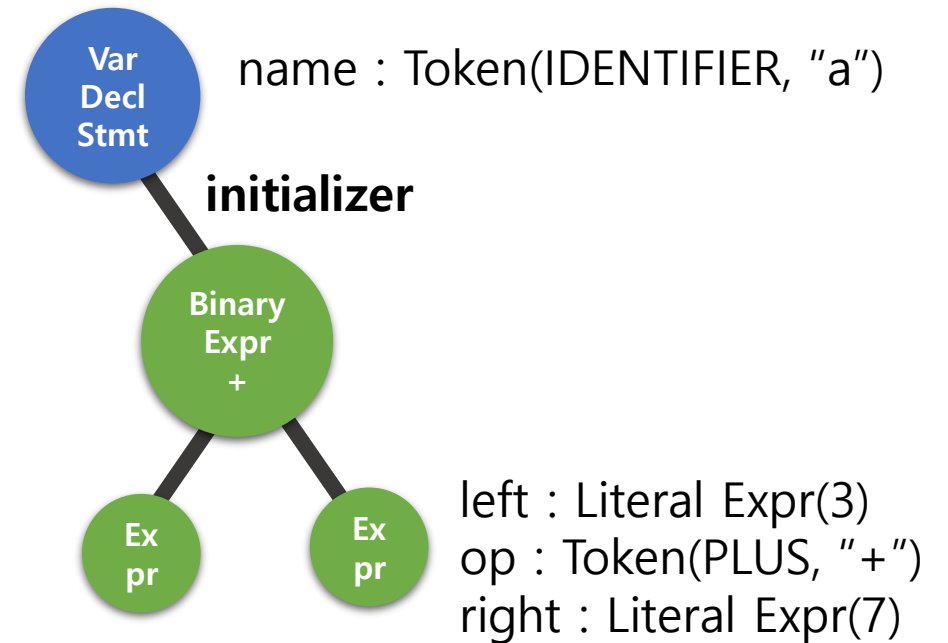
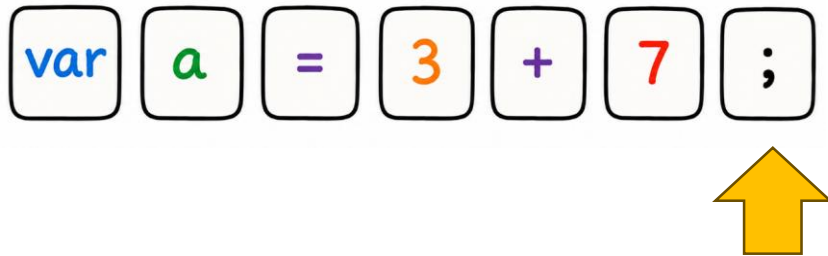
Binary Expr 생성후 조립 및 연결

Binary Expr 노드를 생성하고, Var Declare Stmt의 initializer 로 입력한다.



Assembly Unit 핵심원리 – 조립과정 8

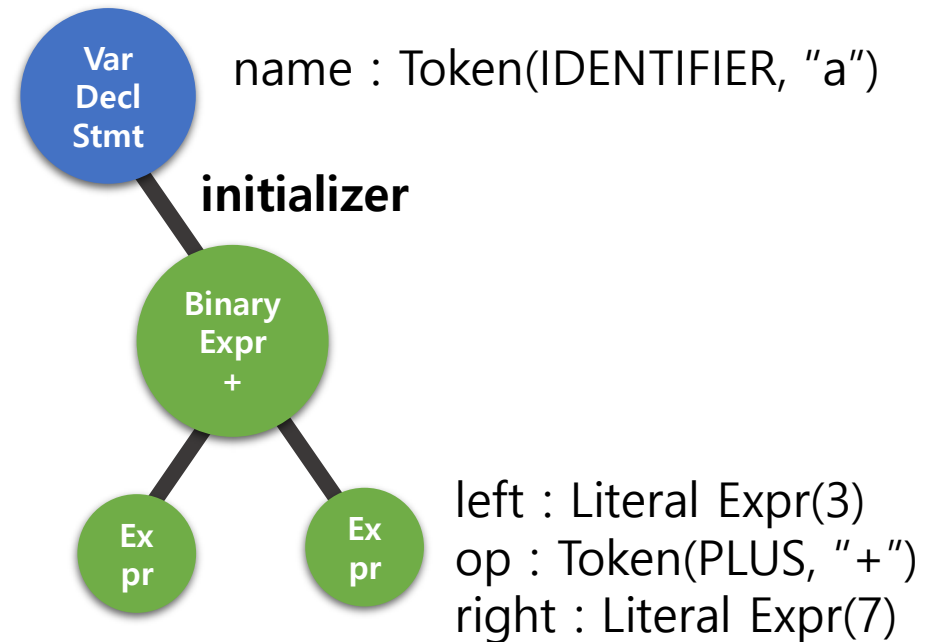
SEMICOLON 발견 시 변수선언문의 parsing 을 종료하고
Var Declare Stmt 을 최종 생성한다.



Assembly Unit 핵심원리 – 완성

멋지게 만들어졌다. 이후 다른 Unit 에 트리 조립체를 전달한다.

- Checker Unit 에서는 트리 조립체를 통해 실행전 오류를 검토하고,
- Executor Unit 에서는 트리 조립체를 실제 실행하여 결과를 얻는다.



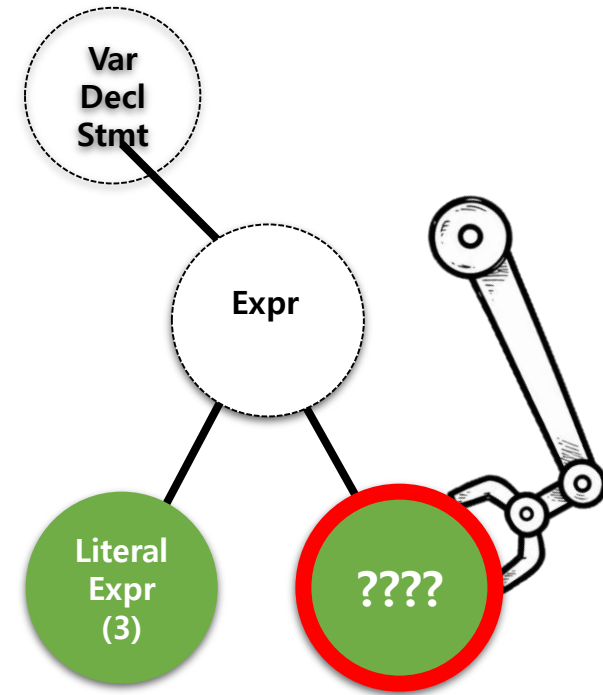
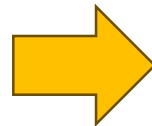
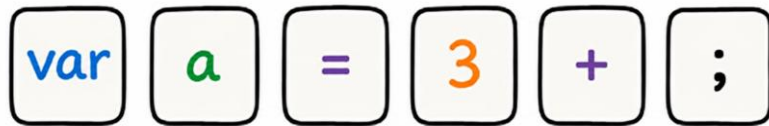
Assembly Unit 핵심원리 – 문법 검사

문법 검사

토큰화된 부품 조립과정에서 올바르지 못한 부품인 경우 오류를 발생시킨다.

예시 : 변수 선언문 코드에서 일부 빠진 경우

var a = 3 + ; ← + 우항부분 빠짐

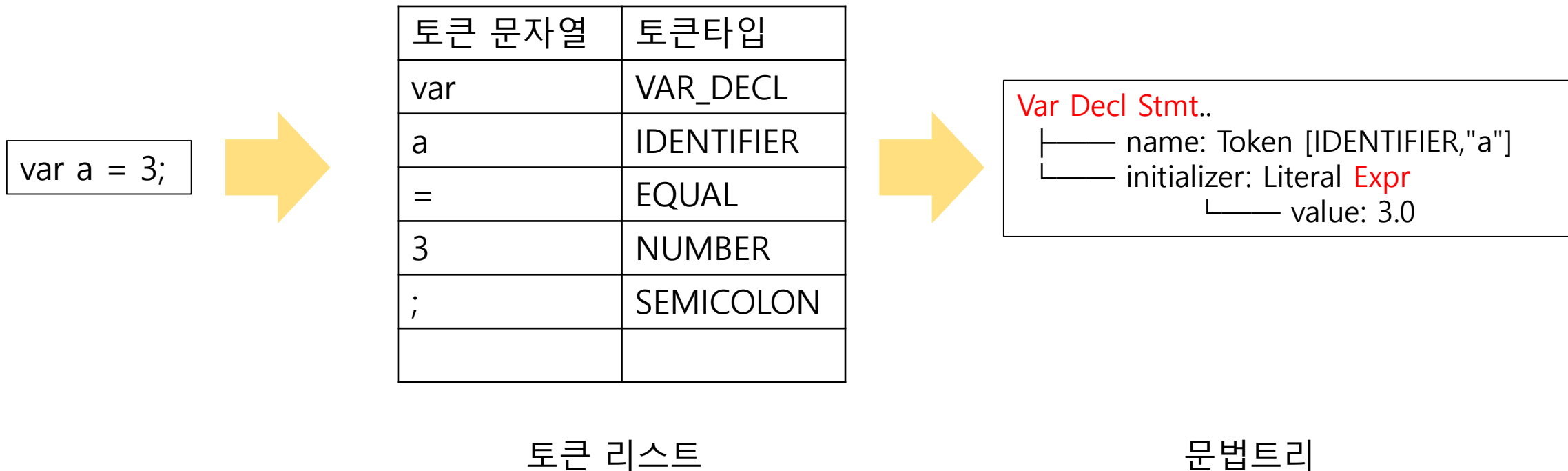


오류 발생

Assembler Unit 요약

원자재 (소스코드) → 부품(Token) → 조립체(Expr Stmt/Token 문법트리)

- 원자재인 소스코드를 분석하여, 부품들 모음인 Token List 를 생성한다.
- Token List 를 읽어서 실행시킬 수 있는 문법 트리를 완성한다.
- 이후, Executor Unit 에서는 문법트리를 실행할 수 있게 된다.



Token, Expr, Stmt 정리

기본적으로 구현해야할 Token, Expr, Stmt 에 대한 안내

- 해당 내용은 참고용으로 필요시 추가, 축소, 변경 가능합니다.
- 문법 규칙의 세부사항은 팀에서 정합니다.
- 따라서 문법 규칙에 적용되는 각 노드에 필요한 field 들은 팀에서 정합니다.
 - 예) Binary Expr 노드의 경우 left(Expr) , right(Expr), operator(어떤연산자인지 - Token 또는 enum) 필드가 필요합니다.
 - 예) 변수 선언문 Var Stmt 노드의 경우 name(변수 이름 토큰), initializer (Expr, 없을수 있음) 필드가 필요합니다.

단, 뒷부분에 주어지는 테스트용 스크립트를 전부 통과 할 수 있어야 합니다.

→ 테스트용 스크립트 문법을 custom 언어에 맞도록 변경 가능

Chapter 03

Checker Unit 제작

조립체를 실행하기 전 검수를 한다

Checker Unit 오류 검사

Checker Unit 은 소스코드의 의미상 오류검사를 수행한다.

- Assembler Unit 이 만든 조립체 검수를 통해 소스코드의 오류, 경고를 검출한다.
- 문법적 오류는 Assembly Unit 에서 수행을 했고, 여기서는 코드간 의미상 오류검사를 한다.

Checker Unit 에서 구현할 아래 3가지에 대해 알아본다.

- 변수 중복 선언 Error 검출/ 지역 변수 초기화 시 자기 참조 Error 검출

[복습]Checker Unit 핵심원리 – 오류 검사

코드간 규칙들이 잘 지켜졌는지 검사

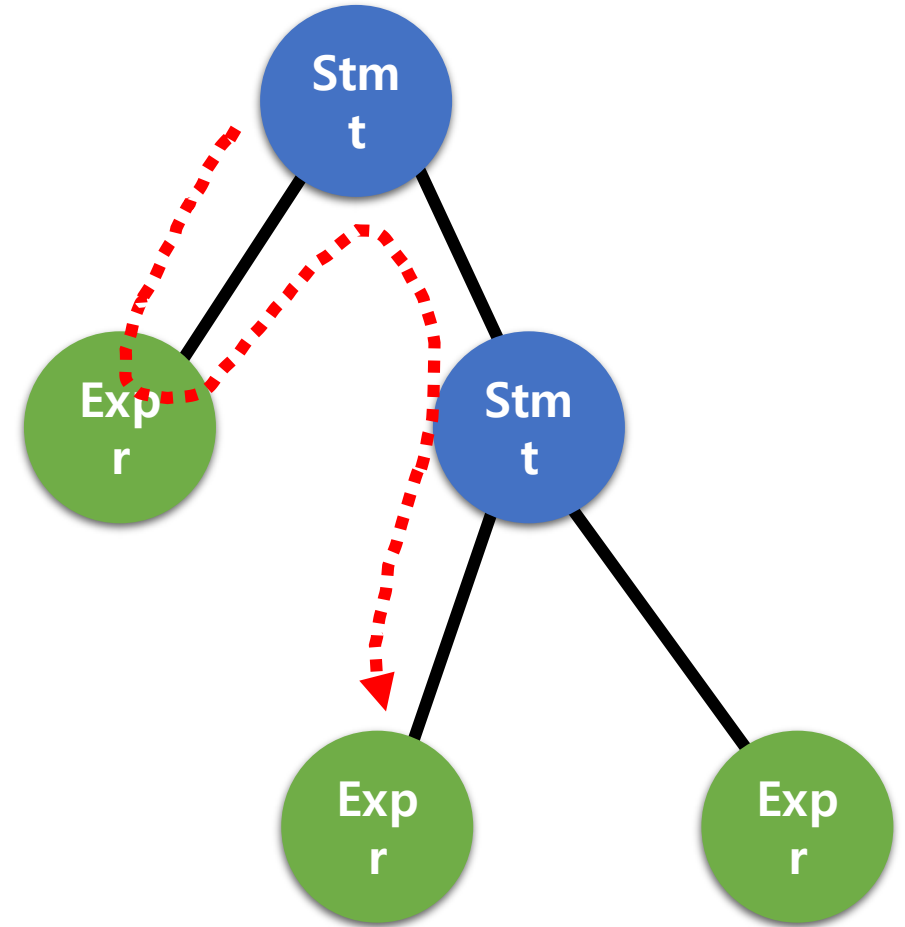
재귀호출로 구현한 DFS 알고리즘을 사용해 각 노드들 탐색

검사하는 것

DFS를 돌면서 각 Stmt, Expr 들이 의미적으로 올바른지 검사

- 변수 중복 선언인지 (같은 블록 내 중복)
- 선언 시 자기 참조

등 코드간의 **의미적인 규칙**을 어겼는지 검사합니다.



오류 검사 – 로컬 스코프 중복 선언 오류

로컬 스코프 내에 발생할 수 있는 논리적 오류를 체크한다.

- Checker Unit 은 현재 스코프에 동일한 변수 선언 시 오류로 처리한다.

```
{  
    var a = "first";  
    var a = "second"; // Error  
}
```

```
> { var a = 10; }  
> { var a = 11; var a = 12; }  
> [1번째 줄] 'a'에러: 이미 해당 변수는 현재 스코프에서 사용중입니다.
```


오류 검사 – 선언시 자기 참조

초기화되지 않은 상태에서 접근하는 오류를 체크한다.

```
{  
    var a = a + 1; // Error  
}
```

> [2번째 줄] 자신의 초기화식에서
지역변수를 읽을 수 없습니다.

선언 시 자기 자신을 참조

Chapter 04

Executor Unit 제작

조립체의 tree 구조를 순회하며 실제로 실행한다

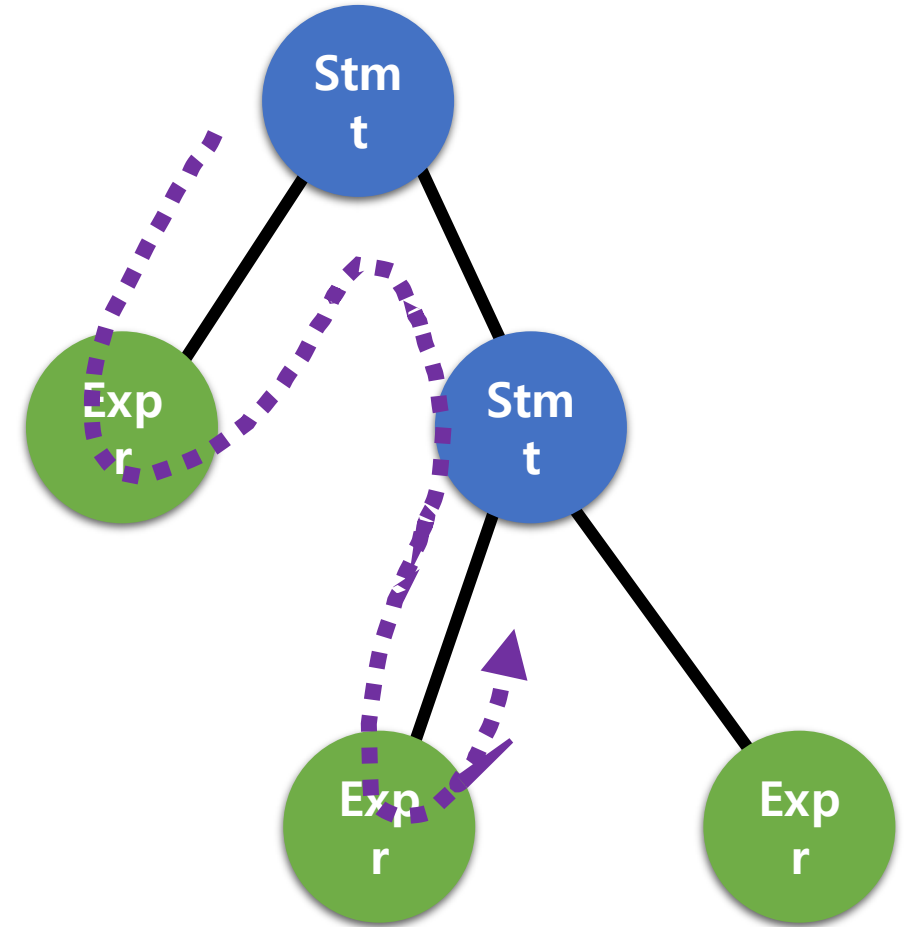
Tree 구조 순회하며 실행

Executor Unit 은 트리구조의 Stmt 조립체를 실행한다.

- 조립체의 트리구조를 재귀적으로 평가하며 실제로 실행한다.

[복습] Executor Unit 핵심원리 - 실행하기

문법 Tree가 다 만들어졌으니,
재귀호출으로 구현된 DFS 알고리즘을 사용하여
각 **Expr**와 **Stmt**를 실행합니다.



실행 예시

예시 : **print** 3 + 2 ;

Print Stmt

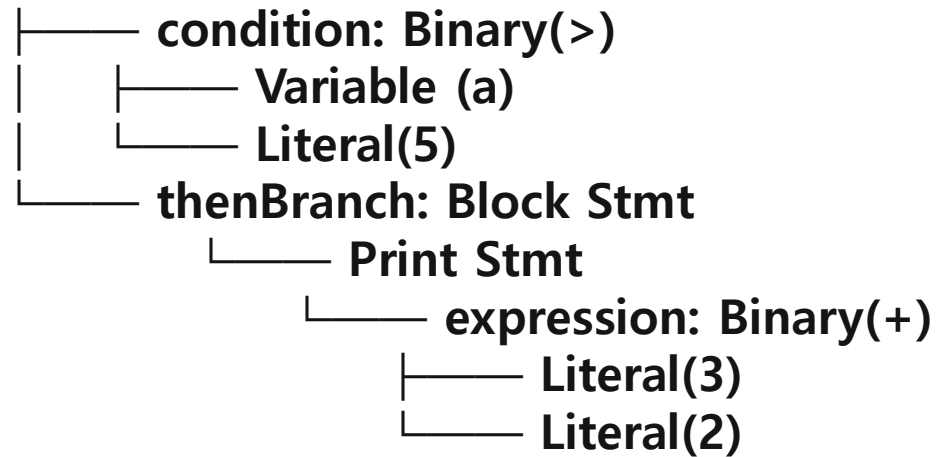
└── **expression: Binary(+)**
 └── **Literal(3)**
 └── **Literal(2)**

1. Print Stmt --> expression 평가 시작
2. Binary (+) --> left, right 순서로 평가
3. Literal (3) --> 3 반환
4. Literal (2) --> 2 반환
5. Binary (+) --> 3 + 2 = 5 반환
6. Print Stmt --> stdout 에 5 출력

실행 예시

예시 : `if ( a > 5 ) { print 3 + 2; }`

If Stmt

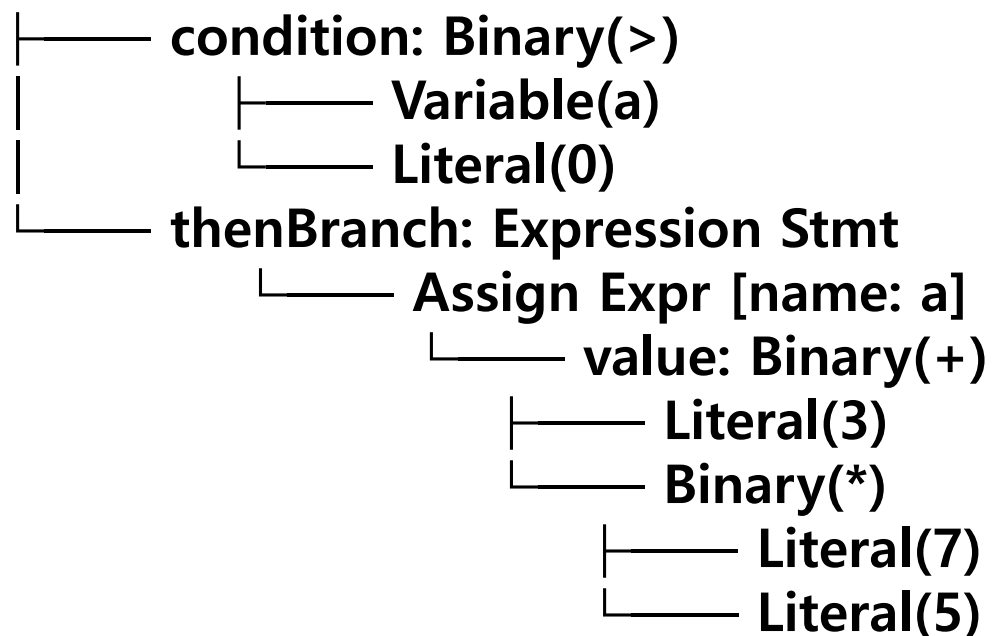


1. If Stmt → condition 평가 시작
2. Variable(a) → a의 값 반환 (예: 10)
3. Literal(5) → 5 반환
4. Binary(>) → $10 > 5 = \text{true}$ 반환
5. If Stmt → true이므로 thenBranch 진입
6. Block Stmt → 내부 문장 실행 시작
7. Print Stmt → expression 평가 시작
8. Literal(3) → 3 반환
9. Literal(2) → 2 반환
10. Binary(+) → $3 + 2 = 5$ 반환
11. Print Stmt → print(5) 출력 완료
12. Block Stmt → 블록 실행 완료
13. If Stmt → 실행 완료

실행 예시

예시: **if** (a > 0) a = 3 + 7 * 5;

If Stmt

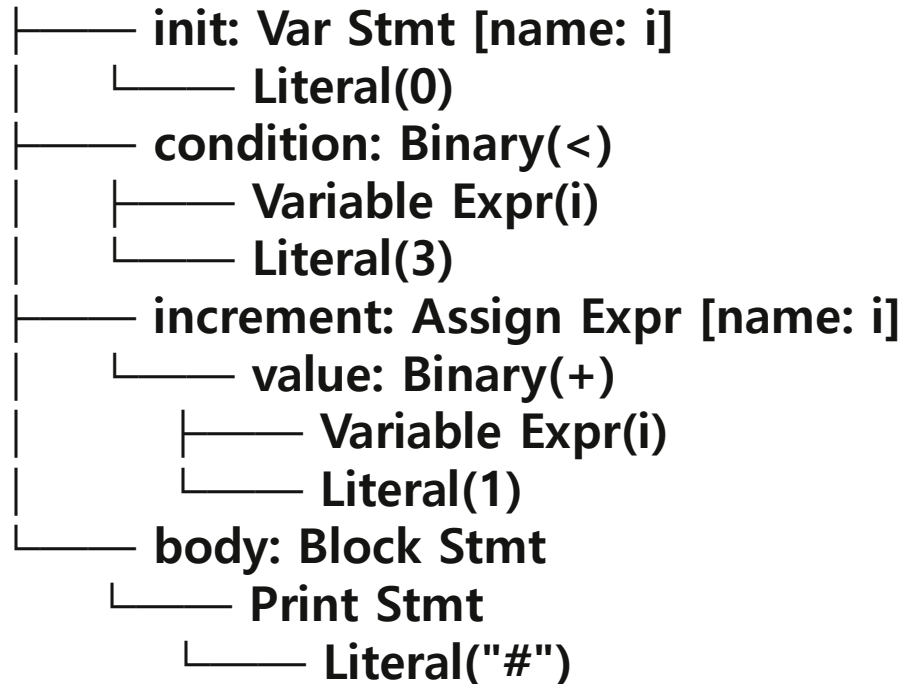


1. If Stmt → condition 평가 시작
2. Variable(a) → a의 값 반환 (예: 10)
3. Literal(0) → 0 반환
4. Binary(>) → $10 > 0 = \text{true}$ 반환
5. If Stmt → true이므로 thenBranch 진입
6. Literal(7) → 7 반환
7. Literal(5) → 5 반환
8. Binary(*) → $7 * 5 = 35$ 반환
9. Literal(3) → 3 반환
10. Binary(+) → $3 + 35 = 38$ 반환
11. Assign Expr → a = 38 저장
12. Expression Stmt → 실행 완료
13. If Stmt → 실행 완료

Tree 구조 순회 예시

예시: `for (var i = 0; i < 3; i = i + 1) { print "#"; }`

For Stmt



초기화

1. For Stmt→초기화 실행
2. Var Stmt→i = 0 선언 및 저장

1회차 (i = 0)

3. For Stmt→condition 평가
4. Variable Expr(i)→0 반환
5. Literal(3)→3 반환
6. Binary(<)→0 < 3 = true 반환
7. For Stmt→true이므로 body 진입
8. Block Stmt→블록 실행 시작
9. Literal("#")→"#" 반환
10. Print Stmt→print("#") 출력
11. Block Stmt→실행 완료
12. For Stmt→increment 실행
13. Variable Expr(i)→0 반환
14. Literal(1)→1 반환
15. Binary(+)→0 + 1 = 1 반환
16. Assign Expr→i = 1 저장

3~16 동일하게 반복 (i = 1 → print "#" → i = 2)

3~16 동일하게 반복 (i = 2 → print "#" → i = 3)

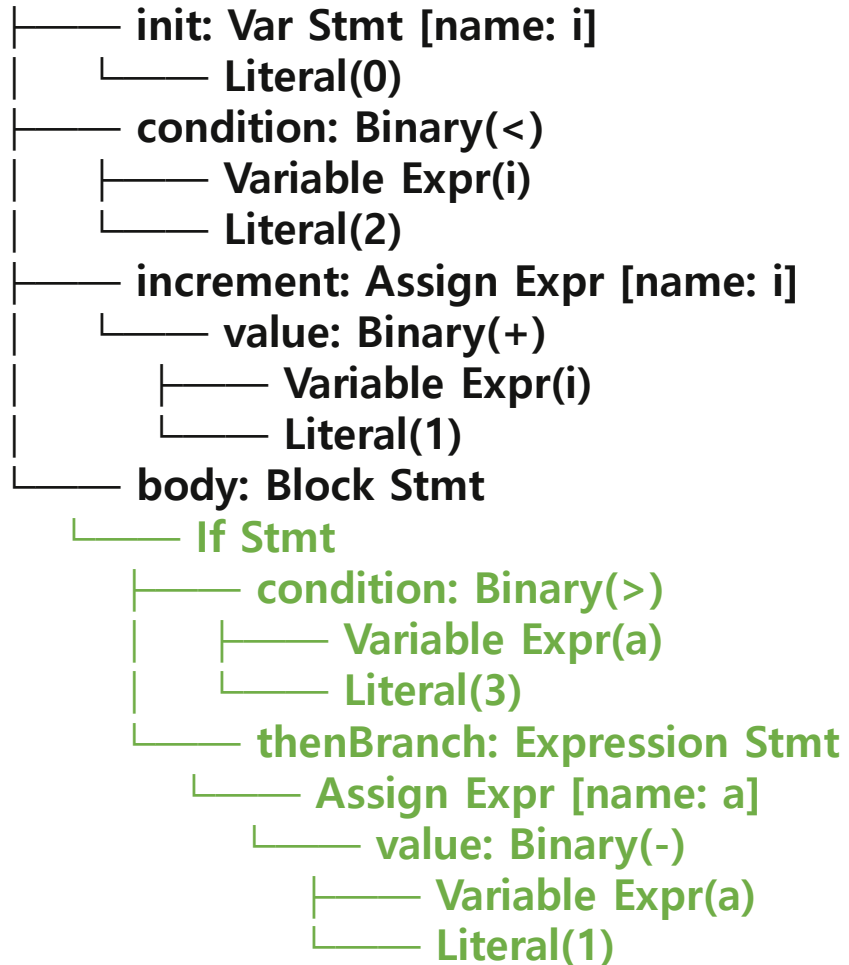
종료 조건 (i = 3)

17. For Stmt→condition 평가
18. Variable Expr(i)→3 반환
19. Literal(3)→3 반환
20. Binary(<)→3 < 3 = false 반환
21. For Stmt→false이므로 루프 종료

Tree 구조 순회 예시

예시: `for (var i = 0; i < 2; i = i + 1) { if(a > 3) a = a - 1; }`

For Stmt



초기화

1. For Stmt→초기화 실행
2. Var Stmt→i = 0 선언 및 저장
- 1회차 (i = 0, a = 5)
3. For Stmt→condition 평가
4. Variable Expr(i)→0 반환
5. Literal(2)→2 반환
6. Binary(<)→0 < 2 = true 반환
7. For Stmt→true이므로 body 진입
8. Block Stmt→블록 실행 시작
9. If Stmt→condition 평가
10. Variable Expr(a)→5 반환
11. Literal(3)→3 반환
12. Binary(>)→5 > 3 = true 반환
13. If Stmt→true이므로 thenBranch 진입
14. Variable Expr(a)→5 반환
15. Literal(1)→1 반환
16. Binary(-)→5 - 1 = 4 반환
17. Assign Expr→a = 4 저장
18. Expression Stmt→실행 완료
19. If Stmt→실행 완료
20. Block Stmt→실행 완료
21. For Stmt→increment 실행
22. Variable Expr(i)→0 반환
23. Literal(1)→1 반환
24. Binary(+)→0 + 1 = 1 반환
25. Assign Expr→i = 1 저장

3~25 동일하게 반복 (i = 1, a = 4 → a = 3 → i = 2)

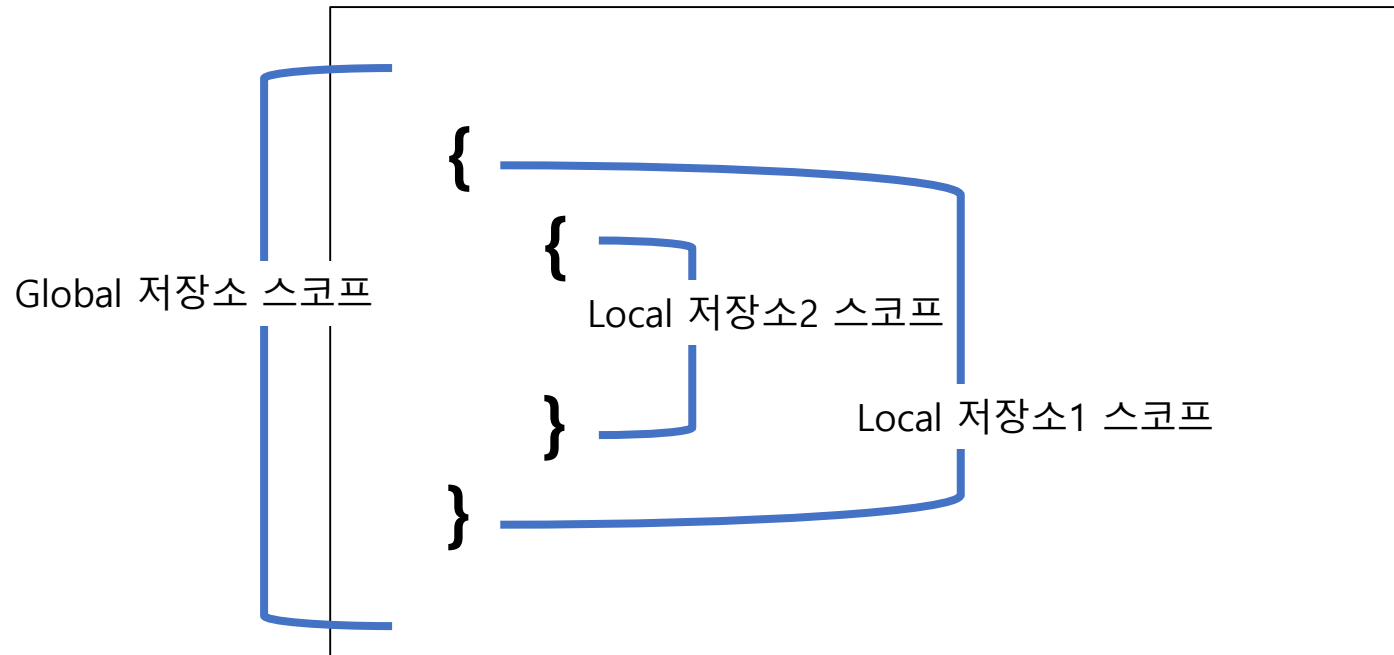
종료 조건 (i = 2)

26. For Stmt→condition 평가
27. Variable Expr(i)→2 반환
28. Literal(2)→2 반환
29. Binary(<)→2 < 2 = false 반환
30. For Stmt→false이므로 루프 종료

변수 저장소

실행중 변수의 이름과 값을 저장하고 참조하는 데이터 저장소 필요

- Executor Unit 은 실행하는 동안 변수 저장소를 운용한다.
- Global 저장소와 Local 저장소로 나뉜다.
- Global 저장소는 전역으로 사용된다.
- Local 저장소는 { } 블록으로 감싼 지역에서 사용된다.



변수 저장소의 생성과 소멸

블록 { }이 생성될 때마다 새로운 로컬 변수 저장소를 생성하고, 블록 종료시 소멸한다.

- **블록 { } 진입** : 새로운 변수 저장소 생성
- **블록 { } 종료** : 현재 변수 저장소 소멸

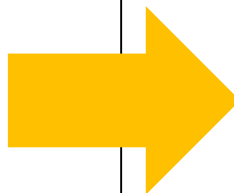
```
var a = 10;      --- global 변수 저장소에 저장
{               --- 해당 로컬 변수 저장소 새로 생성
    var b = 20;  --- 로컬 변수 저장소에 저장
    print a + b; --- a : global에서, b: 로컬 변수 저장소에서 읽기
}               --- 로컬 변수 저장소 소멸
var c = 20;      --- global 변수 저장소에 저장
```

여러 개의 로컬 저장소

변수 사용시, 가장 인접한 저장소부터 상위로 거슬러 올라가며 찾는다.

- 현재 저장소 → 상위 저장소 → 상위 저장소 → → Global 저장소

```
var ga = 3;           -- Global 저장소
{
  var a = 2;          -- 저장소 A
  {
    var a = 7;        -- 저장소 B
    {
      -- 저장소 C
      print a;         // 1
      print ga;        // 2
    }
    print a;           // 3
  }
}
```



1, 2, 3 의 코드에서 변수 저장소 찾는 과정

1. 저장소 C (없음) → 저장소 B (찾음!)
2. 저장소 C (없음) → 저장소 B (없음)
→ 저장소 A (없음) → Global(찾음!)
3. 저장소 B (찾음!)

Executor Unit 은 실행중 다음 Runtime 오류들을 처리한다.

- 타입 불일치
- 정의되지 않은 변수 참조
- 0 으로 나누기

타입 불일치

Executor Unit 은 실행중에 타입 불일치로 연산할 수 없는 경우 런타임 오류를 보고한다.

- 피연산자 타입 오류

예) `true * false` : Boolean 타입에 대해 * 연산은 지원하지 않는다.

예) `3 - "hello"` : 숫자 - 문자열은 불가능하다.

- 이런 경우 어떤 문제 상황으로 오류가 발생한건지 명시해서 오류를 보고한다.

예) `3 - "hello"`

> **[1 번째줄] 피연산자는 반드시 숫자여야 한다.**

정의 되지 않은 변수 참조

Executor Unit 은 실행중에 미정의 변수를 참조시 런타임 오류를 보고한다.

- 선언된적이 없는 변수 참조시 에러
- 예) 선언 없이 $x = 5;$

> [1번째줄] 미정의된 변수 'x'

0으로 나누는 동작 에러

Executor Unit 은 실행중에 0으로 나누는 동작을 할 경우 런타임 오류를 보고한다.

- 나눗셈 식에서 제수(Divisor) 가 0인 경우, 런타임 오류를 발생한다
- 예) $a = 3 / 0 ;$

> [1번째줄] 0으로 나눈 오류

Chapter 05

테스트용 스크립트

테스트용 스크립트 실행시 정상 체크

테스트 스크립트

다음 링크에서 제공되는 예시 스크립트를 통과하도록 구현한다.

단, 스크립트에 등장하는 기능과 동작은 유지한 채로 언어 문법은 변경 가능

<https://gist.github.com/aijeonghwan-star/d1535e870aeb6a4a928142d4d57c191e>

chapter 6

팀 프로젝트 진행

1 ~ 2일차 / 3 ~ 5 일차 프로젝트 진행 안내

프로젝트 준비

- 창의력있는 팀 이름을 정해주세요.
- 팀에서 사용할 코딩규칙 (코드 컨벤션, 코딩 스탠다드 및 룰)을 설정합니다.
- 팀장님과 팀원 분들의 역할을 분배합니다

[1~2일차] 요청사항

Claude Code 사용

Claude Code 를 개발 및 협업에 사용 가능합니다.

빠른 코드 생성이 가능하지만 생성된 코드의 이해 없이, 검토 없이 넘어가면 안됩니다.

TDD 개발 필수

1일차 ~ 2일차 : TDD 개발이 포함되어야 합니다.

3일차 ~ 4일차 : TDD 없이 개발해도 됩니다. Unit Test + 리팩토링만 해주세요.

3일차에는 기능 추가를 요청을 드릴 예정입니다.

- function 관련 기능, 정적 배열 구현, 실행전 최적화 등 추가 기능에 대한 요구가 있습니다
- 기능 추가를 대비하여, 1 ~ 2일차 까지 클린한 코드를 만들어두어야 합니다.
→ 새로운 기능 추가에 대응하기 좋은 디자인 패턴, 설계 등이 필요합니다.

[5일차] 발표 준비 및 발표

- 5일차에는 그 동안 제작했던 프로젝트 발표 준비를 합니다.
오전 : PPT 준비 / 오후 : 발표 및 마무리
팀장님이 발표해주시면 됩니다. (변경하셔도 됩니다.)
- 발표 자료에는 리팩토링 전 후 결과와 코드리뷰활동 캡처가 포함되어야합니다.
클린코드를 신경쓰지 않는 코드를 먼저 만들고,
리팩토링을 통해 점차 개선하는 방식으로 개발을 권장합니다.
리팩토링 전 후 결과 캡처가 가능하도록, 리팩토링 과정마다 Commit을 권장합니다.



CodeFab Interpreter 프로젝트 시작

- CodeFab Interpreter **프로젝트를 시작해주세요**
 - 프로젝트 진행 / 기능 구현 질문
프로젝트 코치 / 강사님에게 언제든지 질문주세요.